HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO
THỰC TẬP CƠ SỞ

ĐỀ TÀI:

ỨNG DỤNG KIẾN TRÚC RAG KẾT HỢP LLMS TRONG XÂY DỰNG CHATBOT HỎI ĐÁP LỊCH SỬ PHỤC VỤ GIÁO DỤC PHỔ THÔNG

Giảng viên hướng dẫn:

Sinh viên thực hiện:

Mã sinh viên:

Lớp:

Khóa:

Hệ: **ĐẠI HỌC CHÍNH QUY**

HÀ NỘI – 5/2025

NHẬN XÉT, ĐÁNH GIÁ, CHO ĐIỂM

(Của giảng viên hướng dẫn)

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Điểm: (Bằng chữ:)

Hà Nội, ngày … tháng 6 năm 2025

Giảng viên hướng dẫn

LỜI MỞ ĐẦU

Trong kỷ nguyên số hóa mạnh mẽ hiện nay, việc ứng dụng công nghệ thông tin và trí tuệ nhân tạo (AI) vào mọi lĩnh vực của đời sống xã hội, đặc biệt là giáo dục, đã trở thành một xu thế tất yếu. Môn Lịch sử, với vai trò quan trọng trong việc giáo dục truyền thống, bồi đắp lòng yêu nước và hình thành thế giới quan cho thế hệ trẻ, cũng không nằm ngoài xu thế đó. Tuy nhiên, phương pháp giảng dạy và học tập Lịch sử truyền thống đôi khi còn gặp phải những thách thức như tính khô khan, khó nhớ của kiến thức, sự thiếu hụt các công cụ tương tác trực quan, dẫn đến việc chưa thực sự khơi gợi được niềm đam mê và hứng thú cho học sinh. Trước bối cảnh đó, đề tài "Ứng dụng kiến trúc RAG kết hợp LLMs trong xây dựng Chatbot hỏi đáp lịch sử phục vụ giáo dục phổ thông" được thực hiện với mong muốn góp phần đổi mới phương pháp tiếp cận môn Lịch sử, mang đến một công cụ hỗ trợ học tập hiện đại, thông minh và hiệu quả hơn cho học sinh phổ thông.

Sự phát triển vượt bậc của các Mô hình Ngôn ngữ Lớn (Large Language Models - LLMs) trong những năm gần đây đã mở ra những tiềm năng to lớn trong việc xử lý và hiểu ngôn ngữ tự nhiên, tạo ra các ứng dụng AI có khả năng tương tác và cung cấp thông tin một cách thông minh. Tuy nhiên, LLMs vẫn còn tồn tại những hạn chế như hiện tượng "ảo giác" (hallucination) – đưa ra thông tin không chính xác hoặc không có trong dữ liệu huấn luyện, và kiến thức bị giới hạn bởi thời điểm huấn luyện. Để khắc phục những nhược điểm này, kiến trúc Truy xuất Kết hợp Sinh văn bản (Retrieval Augmented Generation - RAG) đã ra đời. RAG cho phép LLMs truy cập và sử dụng thông tin từ một kho tri thức bên ngoài (corpus) cụ thể trong quá trình sinh câu trả lời, từ đó tăng cường tính chính xác, độ tin cậy và khả năng cập nhật kiến thức cho hệ thống. Việc kết hợp RAG và LLMs, đặc biệt là các mô hình tiên tiến như Gemini (Google Deepmind), hứa hẹn mang lại một giải pháp đột phá cho việc xây dựng các hệ thống hỏi đáp chuyên sâu, đặc biệt phù hợp với các lĩnh vực đòi hỏi tính chính xác cao như Lịch sử. Framework LightRAG, một giải pháp

RAG tiên tiến được thiết kế chuyên biệt cho ngôn ngữ tiếng Việt với kiến trúc tri thức đồ thị, được lựa chọn làm nền tảng để phát triển ứng dụng trong đề tài này.

Lời cuối, em xin chân thành cảm ơn Thầy X đã tận tình hướng dẫn, định hướng và cung cấp những kiến thức quý báu để em có thể hoàn thành báo cáo thực tập cơ sở này. Mặc dù đã rất cố gắng, nhưng do kiến thức và kinh nghiệm còn hạn chế, báo cáo khó tránh khỏi những thiếu sót. Em rất mong nhận được những ý kiến đóng góp quý báu từ Thầy để đề tài được hoàn thiện hơn.

MỤC LỤC

DANH MỤC HÌNH ẢNH 8
DANH MỤC TỪ VIẾT TẮT 9
CHƯƠNG 1: GIỚI THIỆU TỔNG QUAN 10
1.1. Đặt vấn đề và lý do chọn đề tài 10
1.2. Mục tiêu nghiên cứu của báo cáo 10
1.3. Tổng quan về kiến trúc RAG (Retrieval Augmented Generation) 11
1.3.1. Khái niệm và sự cần thiết của RAG 11
1.3.2. Các thành phần chính và luồng hoạt động cơ bản của RAG 12
1.4. Giới thiệu về Framework LightRAG 14
1.4.1. LightRAG là gì? Mục tiêu và triết lý thiết kế 14
1.4.2. Các ưu điểm nổi bật của LightRAG 16
1.4.3. Sơ đồ kiến trúc tổng quan của LightRAG 17
1.4.4. Luồng xây dựng kho tri thức của LightRAG 19
CHƯƠNG 2: PHÂN TÍCH CHI TIẾT CÁC THÀNH PHẦN VÀ QUY TRÌNH HOẠT ĐỘNG CỦA LIGHTRAG 22
2.1. Kiến trúc tổng quan của LightRAG 22
2.2. Kiến trúc lưu trữ đa tầng 23
2.2.1. Storage Abstraction 23
2.2.2. Document Status Tracking 24
2.3. Xử lý ngôn ngữ tiếng Việt 24
2.3.1. Entity Types được tối ưu cho lịch sử Việt Nam 25
2.3.2. Vietnamese LLM Wrapper 25
2.4. Knowledge Graph Operations 26
2.4.1. Entity và Relationship Extraction 28
2.4.2. Graph Storage và Query 29
2.5. Query Modes 29

2.6. Context Support 30
2.7. Document Management 31
2.7.1. Insertion Pipeline 31
2.7.2. Deletion Capabilities 31
2.8. Tính năng nâng cao 31
2.8.1. Custom Knowledge Graph Import 32
2.8.2. Batch Processing 32
2.8.3. Entity Information Retrieval 32
CHƯƠNG 3: MINH HỌA ỨNG DỤNG LIGHTRAG TRONG XÂY DỰNG CHATBOT LỊCH SỬ 34
3.1. Bối cảnh bài toán ứng dụng (Chatbot hỏi đáp Lịch sử) 34
3.2. Lựa chọn và cấu hình các thành phần LightRAG cho bài toán 35
3.2.1. Lựa chọn Document Loader, Text Splitter, Embedder phù hợp với dữ liệu Lịch sử 35
3.2.2. Cấu hình VectorDB và các tham số cho Retriever 36
3.2.3. Thiết kế prompt cho LLM (Gemini) 38
3.3. Phân tích kết quả thử nghiệm 41
3.3.1. Minh họa khả năng truy xuất và sinh câu trả lời 41
3.3.2. Đánh giá sơ bộ hiệu quả 43
3.4. Demo ứng dụng 44
3.4.1. Các tính năng chính của ứng dụng 44
3.4.2. Một số hình ảnh từ ứng dụng được cài đặt 45
CHƯƠNG 4: KẾT LUẬN VÀ ĐỀ XUẤT 53
4.1. Tổng kết những phân tích chính về các thành phần của LightRAG 53
4.1.1. Quy trình Xử lý và Lập chỉ mục Dữ liệu (Data Ingestion and Indexing Pipeline) 53
4.1.2. Quy trình Truy vấn và Sinh câu trả lời (Querying and Generation Pipeline) 54
4.1.3. Evaluators (Thành phần Đánh giá) 54
4.2. Đánh giá chung về ưu điểm, nhược điểm của LightRAG 55

4.3. Thách thức khi làm việc với LightRAG và hệ thống RAG 56
4.4. Đề xuất hướng phát triển (đối với LightRAG framework) 57
4.5. Hướng phát triển đề tài 59
TÀI LIỆU THAM KHẢO 61

DANH MỤC HÌNH ẢNH

Hình 1. Các thành phần chính của RAG 13
Hình 2. Luồng hoạt động cơ bản của RAG 14
Hình 3. Sơ đồ kiến trúc tổng quan của LightRAG 17
Hình 4. Luồng xây dựng kho tri thức của LightRAG 19
Hình 5. Cấu trúc thư mục của LightRAG 22
Hình 6. Các loại thực thể phù hợp với lĩnh vực Lịch sử 25
Hình 7. Wrapper Tiếng Việt cho Gemini 26
Hình 8. Ví dụ về Đồ thị tri thức 27
Hình 9. Trực quan hóa đồ thị tri thức 28
Hình 10. Ví dụ mô tả các nút và các cạnh trong đồ thị tri thức 29
Hình 11. Custom Knowledge Graph Import 32
Hình 12. Batch Processing 32
Hình 14. Menu của ứng dụng 45
Hình 15. Giao diện Chatbot 45
Hình 16. Giao diện quản lí tài liệu 46
Hình 17. Giao diện biểu diễn đồ thị tri thức 46
Hình 18. Đồ thị tri thức toàn hệ thống 47
Hình 19. Ví dụ về thực thể trong đồ thị tri thức 47
Hình 20. Ví dụ về thực thể trong đồ thị tri thức 2 48
Hình 21. Ví dụ về thực thể trong đồ thị tri thức 3 48
Hình 22. Câu hỏi minh họa 49
Hình 23. Câu hỏi minh họa 2 50
Hình 24. Câu hỏi minh họa 3 51
Hình 25. Câu hỏi minh họa 4 52

DANH MỤC TỪ VIẾT TẮT

STT	Từ viết tắt	Tiếng Anh	Tiếng Việt
1	AI	Artificial Intelligence	Trí Tuệ Nhân Tạo
2	API	Application Programming Interface	Giao diện Lập trình Ứng dụng
3	KV Store	Key-Value Store	Kho lưu trữ Khóa-Giá trị
4	KG	Knowledge Graph	Đồ thị Tri thức
5	LLM	Large Language Model	Mô Hình Ngôn Ngữ Lớn
6	NER	Named Entity Recognition	Nhận dạng Thực thể Có tên
7	NLP	Natural Language Processing	Xử lý Ngôn ngữ Tự nhiên
8	PDF	Portable Document Format	Định dạng Tài liệu Di động
9	RAG	Retrieval Augmented Generation	Truy xuất Kết hợp Sinh văn bản
10	RE	Relation Extraction	Trích xuất Mối quan hệ
11	VectorDB	Vector Database	Cơ sở dữ liệu Vector

CHƯƠNG 1: GIỚI THIỆU TỔNG QUAN

1.1. Đặt vấn đề và lý do chọn đề tài

Sự phát triển vượt bậc của các Mô hình Ngôn ngữ Lớn (LLMs) đã mang lại những khả năng xử lý ngôn ngữ tự nhiên chưa từng có. Tuy nhiên, việc ứng dụng LLMs vào các bài toán thực tế, đặc biệt là các lĩnh vực đòi hỏi độ chính xác cao và kiến thức chuyên ngành (như giáo dục, y tế, pháp luật, ...), bộc lộ những hạn chế như nguy cơ sinh thông tin sai lệch ("ảo giác") và kiến thức bị giới hạn bởi dữ liệu huấn luyện. Kiến trúc Truy xuất Kết hợp Sinh văn bản (RAG) ra đời như một giải pháp tiên tiến, cho phép LLMs truy cập và tận dụng các nguồn tri thức bên ngoài để nâng cao chất lượng câu trả lời.

Trong bối cảnh đó, LightRAG - một framework RAG tiên tiến được thiết kế chuyên biệt cho tiếng Việt với khả năng xử lý tri thức đồ thị - đóng vai trò quan trọng trong việc đơn giản hóa và chuẩn hóa quy trình xây dựng ứng dụng RAG. Việc hiểu rõ cấu trúc độc đáo, các thành phần chính và cơ chế hoạt động của LightRAG là cần thiết để khai thác tối đa tiềm năng của kiến trúc này cho các ứng dụng chuyên sâu, đặc biệt trong lĩnh vực lịch sử Việt Nam. Đề tài này được thực hiện với mong muốn giới thiệu về LightRAG, từ đó cung cấp nền tảng kiến thức cho việc xây dựng các hệ thống RAG hiệu quả và đáng tin cậy, đặc biệt là trong các ứng dụng đòi hỏi tính chuyên biệt cao.

1.2. Mục tiêu nghiên cứu của báo cáo

- Giới thiệu các thành phần cốt lõi của framework LightRAG, bao gồm các module liên quan đến xử lý dữ liệu, lập chỉ mục, truy xuất thông tin, và tích hợp với các mô hình ngôn ngữ lớn.
- Giới thiệu kiến trúc tri thức đồ thị (Knowledge Graph) của LightRAG và cách nó được sử dụng để biểu diễn và truy vấn mối quan hệ giữa các thực thể lịch sử.
- Làm rõ quy trình hoạt động và luồng dữ liệu bên trong LightRAG trong giai đoạn xây dựng kho tri thức và giai đoạn xử lý truy vấn.

- Làm rõ cách LightRAG tối ưu hóa việc xử lý và sinh văn bản cho ngôn ngữ tiếng Việt thông qua các loại thực thể (Entity Types) được định nghĩa sẵn và các wrapper LLM chuyên biệt.
- Đánh giá sơ bộ ưu nhược điểm của LightRAG dựa trên cấu trúc và các tính năng được cung cấp.
- Minh họa khả năng ứng dụng của LightRAG thông qua một bài toán thực tế là xây dựng chatbot hỏi đáp kiến thức Lịch sử.

1.3. Tổng quan về kiến trúc RAG (Retrieval Augmented Generation)

1.3.1. Khái niệm và sự cần thiết của RAG

Kiến trúc Truy xuất Kết hợp Sinh văn bản (RAG) là một phương pháp trong lĩnh vực Xử lý Ngôn ngữ Tự nhiên (NLP) kết hợp sức mạnh của các mô hình truy xuất thông tin (retrieval models) và các mô hình sinh văn bản (generative models), điển hình là các LLM. Thay vì chỉ dựa vào kiến thức nội tại được "nhớ" trong quá trình huấn luyện, các hệ thống RAG đầu tiên truy xuất các đoạn văn bản hoặc tài liệu liên quan từ một kho tri thức lớn (corpus) dựa trên câu hỏi hoặc ngữ cảnh đầu vào. Sau đó, những thông tin được truy xuất này sẽ được cung cấp làm ngữ cảnh bổ sung cho LLM để sinh ra câu trả lời cuối cùng.

Sự cần thiết của RAG xuất phát từ những hạn chế của LLMs truyền thống:

- **Kiến thức lỗi thời:** LLMs thường được huấn luyện trên một tập dữ liệu cố định tại một thời điểm, khiến chúng không thể cập nhật thông tin mới phát sinh sau đó.
- **Hiện tượng "ảo giác" (Hallucination):** LLMs có thể tự tin sinh ra những thông tin không chính xác, không có cơ sở thực tế, đặc biệt khi đối mặt với các câu hỏi nằm ngoài phạm vi kiến thức hoặc các truy vấn phức tạp.
- **Thiếu khả năng giải thích (Explainability):** Khó có thể truy xuất nguồn gốc thông tin mà LLM sử dụng để đưa ra câu trả lời.

- **Khó tùy chỉnh cho kiến thức chuyên ngành:** Việc huấn luyện lại hoặc tinh chỉnh LLMs cho các lĩnh vực chuyên biệt thường tốn kém và phức tạp.

RAG giải quyết các vấn đề này bằng cách:

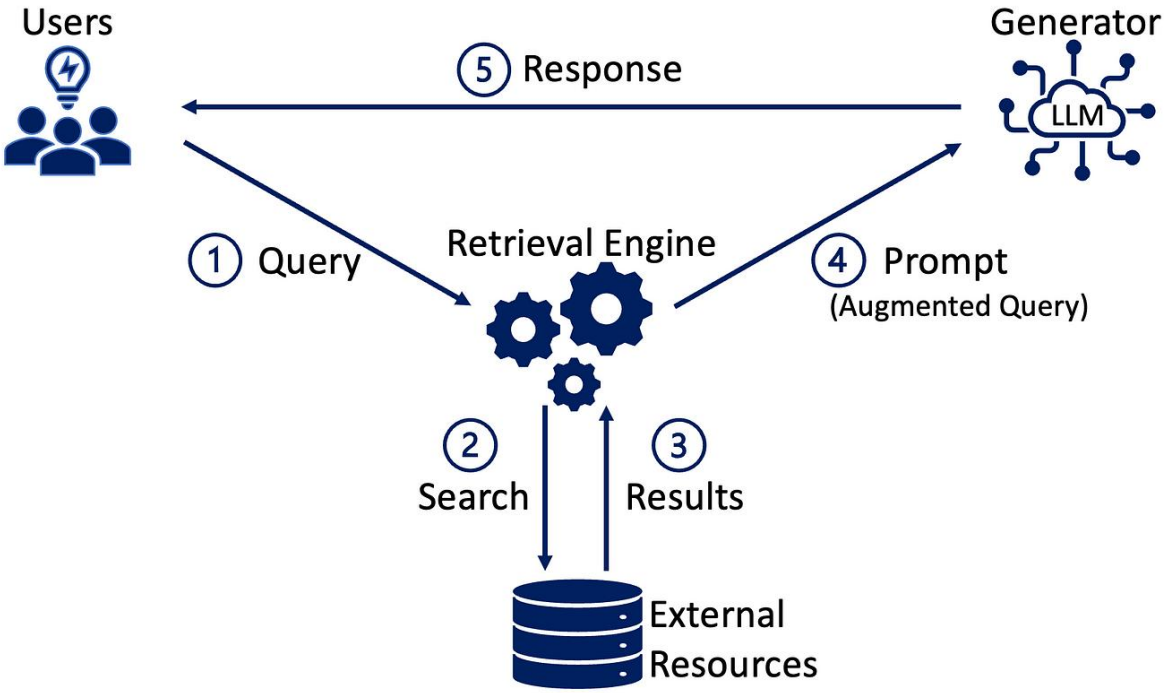
- Cho phép LLMs truy cập kiến thức cập nhật từ các nguồn bên ngoài.
- Giảm thiểu "ảo giác" bằng cách "neo" câu trả lời vào các bằng chứng được truy xuất.
- Tăng khả năng giải thích bằng cách chỉ ra các nguồn thông tin đã được sử dụng.
- Dễ dàng tùy chỉnh hệ thống cho các lĩnh vực cụ thể bằng cách thay đổi hoặc cập nhật kho tri thức.

1.3.2. Các thành phần chính và luồng hoạt động cơ bản của RAG

Một hệ thống RAG điển hình bao gồm hai thành phần chính:

1. **Thành phần Truy xuất (Retriever):**
    - **Nhiệm vụ:** Tìm kiếm và trích xuất các đoạn văn bản (context) hoặc tài liệu có liên quan nhất từ một kho tri thức lớn (ví dụ: cơ sở dữ liệu văn bản, Wikipedia, tài liệu nội bộ) dựa trên câu hỏi hoặc truy vấn đầu vào của người dùng.
    - **Cơ chế hoạt động:** Thường sử dụng các kỹ thuật như tạo vector nhúng (embeddings) cho cả truy vấn và các tài liệu trong kho tri thức, sau đó tìm kiếm sự tương đồng ngữ nghĩa (ví dụ: sử dụng cosine similarity) để lấy ra các kết quả phù hợp nhất.
2. **Thành phần Sinh văn bản (Generator):**

- **Nhiệm vụ:** Tiếp nhận câu hỏi gốc của người dùng cùng với các đoạn văn bản ngữ cảnh liên quan do Retriever cung cấp, sau đó tổng hợp thông tin và sinh ra một câu trả lời cuối cùng, mạch lạc và chính xác.
- **Cơ chế hoạt động:** Thường là một Mô hình Ngôn ngữ Lớn (LLM) đã được huấn luyện trước. LLM sử dụng ngữ cảnh được cung cấp để định hướng và làm giàu cho quá trình sinh văn bản.

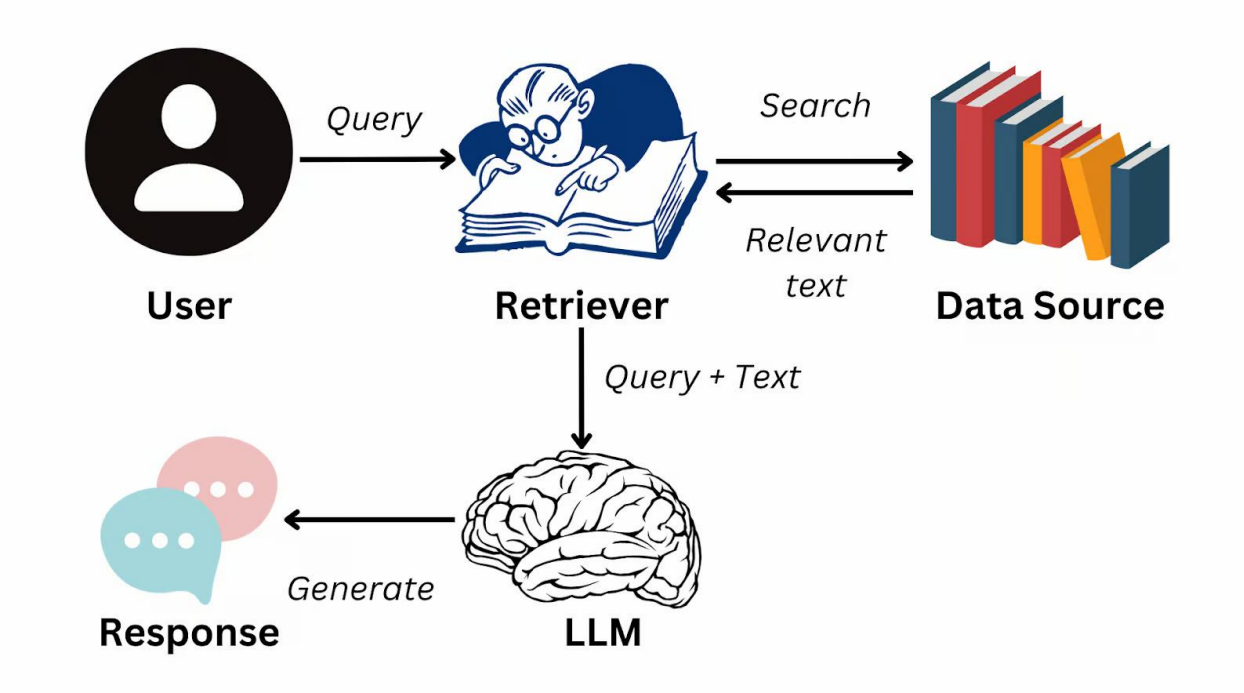


Hình 1. Các thành phần chính của RAG

Luồng hoạt động cơ bản của RAG:

1. **Tiếp nhận truy vấn:** Người dùng nhập câu hỏi.
2. **Truy xuất thông tin:** Retriever xử lý câu hỏi, tìm kiếm và lấy ra các đoạn văn bản ngữ cảnh liên quan từ kho tri thức.
3. **Chuẩn bị đầu vào cho Generator:** Câu hỏi gốc và các ngữ cảnh truy xuất được kết hợp thành một prompt hoàn chỉnh.
4. **Sinh câu trả lời:** Generator (LLM) xử lý prompt và tạo ra câu trả lời.

5. **Trả kết quả:** Câu trả lời được hiển thị cho người dùng.



Hình 2. Luồng hoạt động cơ bản của RAG

1.4. Giới thiệu về Framework LightRAG

1.4.1. LightRAG là gì? Mục tiêu và triết lý thiết kế

LightRAG là một framework mã nguồn mở được thiết kế để đơn giản hóa và tăng tốc quá trình xây dựng các ứng dụng dựa trên kiến trúc Truy xuất Kết hợp Sinh văn bản (RAG), đặc biệt tối ưu cho ngôn ngữ tiếng Việt và sử dụng kiến trúc tri thức đồ thị (Knowledge Graph) để biểu diễn và truy vấn thông tin phức tạp. Tên gọi "Light" (nhẹ) gợi ý đến mục tiêu của framework là cung cấp một giải pháp gọn nhẹ, linh hoạt và dễ sử dụng, giúp các nhà phát triển có thể nhanh chóng tạo mẫu và triển khai các hệ thống RAG mà không cần phải xây dựng lại toàn bộ pipeline từ đầu.

Mục tiêu chính của LightRAG:

- Đơn giản hóa việc xây dựng RAG: Cung cấp các module và giao diện lập trình ứng dụng (API) trực quan để người dùng dễ dàng kết nối các thành

phần khác nhau của một pipeline RAG (như tải dữ liệu, phân đoạn văn bản, tạo nhúng, lưu trữ vector, truy xuất, và sinh văn bản).

- Xây dựng và quản lý đồ thị tri thức: Cung cấp các công cụ để trích xuất thực thể, mối quan hệ và xây dựng đồ thị tri thức từ văn bản tiếng Việt.
- Tối ưu hóa cho ngôn ngữ tiếng Việt: Đảm bảo khả năng xử lý ngôn ngữ tự nhiên tiếng Việt hiệu quả thông qua các định nghĩa loại thực thể chuyên biệt và wrapper LLM tùy chỉnh.
- Tăng tính module hóa và linh hoạt: Cho phép người dùng dễ dàng tùy chỉnh, thay thế hoặc mở rộng các thành phần trong pipeline (ví dụ: sử dụng các mô hình nhúng khác nhau, các cơ sở dữ liệu vector khác nhau, hoặc các LLM khác nhau).
- Hỗ trợ tích hợp nhanh chóng: Tạo điều kiện thuận lợi cho việc tích hợp với các công cụ và dịch vụ phổ biến trong hệ sinh thái AI/NLP.
- Tập trung vào hiệu quả và khả năng mở rộng cơ bản: Cung cấp một nền tảng vững chắc để xây dựng các ứng dụng RAG có thể hoạt động hiệu quả cho nhiều trường hợp sử dụng.

Triết lý thiết kế của LightRAG bao gồm:

- Ưu tiên sự đơn giản và dễ tiếp cận: Giúp người dùng mới có thể nhanh chóng làm quen và sử dụng.
- Sử dụng kiến trúc tri thức đồ thị: Nắm bắt các mối quan hệ phức tạp giữa các thực thể để cung cấp ngữ cảnh phong phú và chính xác hơn.
- Hỗ trợ đa chế độ truy vấn: Cung cấp các chiến lược truy vấn khác nhau (local, global, hybrid, naive, mix) để phù hợp với các loại câu hỏi đa dạng.
- Tính cấu hình cao: Cho phép người dùng tinh chỉnh các tham số và hành vi của từng thành phần.

- Khả năng mở rộng: Thiết kế để có thể dễ dàng bổ sung các tính năng mới hoặc tích hợp các công nghệ mới.

1.4.2. Các ưu điểm nổi bật của LightRAG

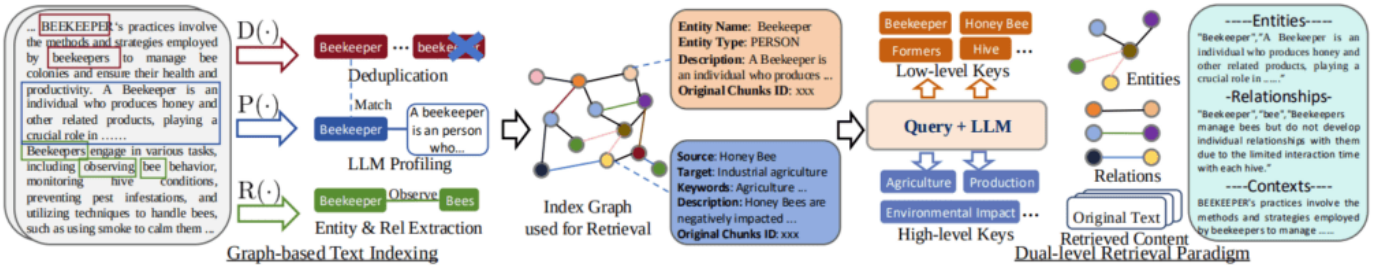
Dựa trên mục tiêu và triết lý thiết kế, LightRAG mang lại một số ưu điểm nổi bật cho người dùng:

- Giảm thời gian phát triển: Thay vì phải tự xây dựng và kết nối từng thành phần của RAG, người dùng có thể tận dụng các module được đóng gói sẵn trong LightRAG, giúp rút ngắn đáng kể thời gian từ ý tưởng đến sản phẩm hoạt động.
- Khả năng hiểu mối quan hệ phức tạp: Nhờ kiến trúc đồ thị tri thức, LightRAG có thể truy vấn và tổng hợp thông tin dựa trên các mối liên hệ ngữ nghĩa sâu sắc giữa các thực thể, vượt xa khả năng tìm kiếm từ khóa hoặc vector đơn thuần.
- Tối ưu hóa cho tiếng Việt: Prompts và các loại thực thể được thiết kế riêng cho ngôn ngữ tiếng Việt đảm bảo chất lượng phản hồi cao và văn phong tự nhiên.
- Các chế độ truy vấn đa dạng: Hỗ trợ nhiều chiến lược truy vấn khác nhau (Local, Global, Hybrid, Naive, Mix Mode) giúp nhà phát triển linh hoạt lựa chọn phương pháp phù hợp nhất với từng loại câu hỏi.
- Dễ sử dụng và triển khai: Với API được thiết kế tốt và tài liệu hướng dẫn, việc học và sử dụng LightRAG trở nên dễ dàng hơn, ngay cả với những người không có quá nhiều kinh nghiệm chuyên sâu về RAG.
- Linh hoạt trong việc lựa chọn thành phần: LightRAG cho phép người dùng lựa chọn và tích hợp các mô hình nhúng, cơ sở dữ liệu vector, và LLM khác nhau tùy theo nhu cầu cụ thể của bài toán và nguồn lực sẵn có.
- Tập trung vào logic nghiệp vụ: Bằng cách trừu tượng hóa các chi tiết kỹ thuật phức tạp của pipeline RAG, LightRAG giúp người dùng tập trung

hơn vào việc xây dựng kho tri thức chất lượng, thiết kế prompt hiệu quả, và tối ưu hóa trải nghiệm người dùng cho ứng dụng cụ thể của họ.

- Quản lý tài liệu nâng cao: Khả năng theo dõi trạng thái xử lý tài liệu (PENDING, PROCESSING, PROCESSED, FAILED) và các chức năng xóa theo ID tài liệu hoặc tên thực thể.
- Cộng đồng hỗ trợ: Là một framework mã nguồn mở, LightRAG có tiềm năng xây dựng một cộng đồng người dùng và nhà phát triển tích cực, cung cấp sự hỗ trợ, chia sẻ kinh nghiệm và đóng góp vào sự phát triển của framework.

1.4.3. Sơ đồ kiến trúc tổng quan của LightRAG



Hình 3. Sơ đồ kiến trúc tổng quan của LightRAG

Quy trình xử lí tài liệu của LightRAG được chia thành 2 giai đoạn:

Giai đoạn 1: Lập chỉ mục văn bản dựa trên đồ thị

Giai đoạn này bắt đầu với một đoạn văn bản thô (.txt, .docx, .pdf, ...). Từ đoạn văn bản này, ba loại xử lý được thực hiện song song:

- **Deduplication (D(-))**: Quá trình này nhận diện và loại bỏ các thực thể trùng lặp. Ví dụ, "Beekeeper" và "Beekeepers" được coi là cùng một thực thể và được chuẩn hóa.
- **LLM Profiling (P(-))**: Sử dụng một Mô hình ngôn ngữ lớn (LLM) để tạo hồ sơ cho các thực thể. Ví dụ, "A beekeeper" được mô tả là "a person who..."

Điều này giúp xác định loại thực thể (ví dụ: PERSON) và cung cấp mô tả chi tiết, cùng với ID của các đoạn văn bản gốc.

- **Entity & Rel Extraction (R(-))**: Trích xuất các thực thể và mối quan hệ giữa chúng. Ví dụ, "Beekeeper" và "Bees" là các thực thể, và "Observe" là mối quan hệ giữa chúng.

Kết quả của ba quá trình này được tổng hợp để tạo ra một "Đồ thị chỉ mục" (Index Graph) được sử dụng cho việc truy xuất. Đồ thị này hiển thị các thực thể (ví dụ: Beekeeper, Honey Bee) và các mối quan hệ giữa chúng, kèm theo thông tin chi tiết như "Target" (mục tiêu), "Keywords" (từ khóa) và "Description" (mô tả).

Giai đoạn 2: Mô hình truy xuất kép

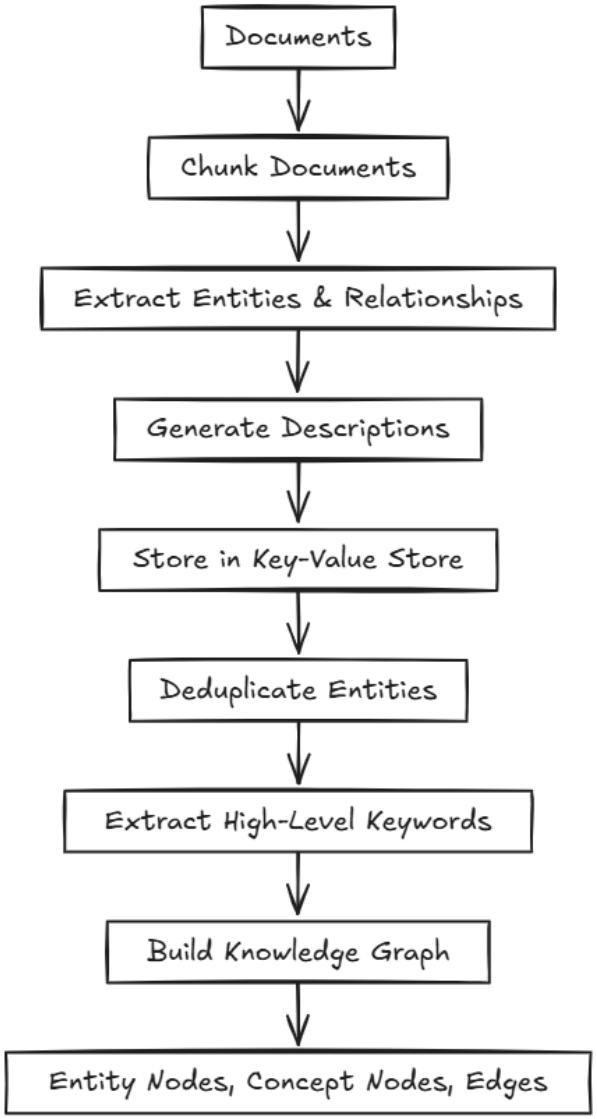
Giai đoạn này sử dụng đồ thị chỉ mục để thực hiện truy xuất thông tin ở hai cấp độ:

- **Query + LLM**: Một truy vấn được đưa vào cùng với LLM để tạo ra các "Khóa cấp thấp" (Low-level Keys) như "Beekeeper", "Honey Bee", "Hive", "Farmers", v.v. và các "Thực thể" (Entities) liên quan.
- **Query + LLM**: Đồng thời, truy vấn cũng được sử dụng để tạo ra các "Khóa cấp cao" (High-level Keys) như "Agriculture", "Production", "Environmental Impact", v.v. và các "Mối quan hệ" (Relations).

Cả hai cấp độ khóa này được sử dụng để truy xuất "Nội dung gốc" (Original Text) và "Nội dung đã truy xuất" (Retrieved Content).

Kết quả cuối cùng của quy trình là một danh sách các "Thực thể" (Entities), "Mối quan hệ" (Relationships) và "Ngữ cảnh" (Contexts) được trích xuất và định nghĩa rõ ràng. Ví dụ, "Beekeeper" được định nghĩa là một cá nhân sản xuất mật ong, và các mối quan hệ như "Beekeeper" và "Bees" được mô tả. Ngữ cảnh cung cấp bối cảnh tổng thể từ văn bản gốc.

1.4.4. Luồng xây dựng kho tri thức của LightRAG



Hình 4. Luồng xây dựng kho tri thức của LightRAG

Một cách tiếp cận khác cho quá trình xây dựng đồ thị tri thức của LightRAG, đó là tuần tự hóa các bước thực hiện:

1. **Tài liệu (Documents)**: Đây là điểm bắt đầu của quy trình, đại diện cho tập hợp các tài liệu đầu vào cần được xử lý.

2. **Phân đoạn Tài liệu (Chunk Documents)**: Các tài liệu lớn được chia thành các đoạn nhỏ hơn (chunks). Bước này giúp việc xử lý sau này hiệu quả hơn và tập trung vào các đơn vị thông tin cụ thể.
3. **Trích xuất Thực thể & Mối quan hệ (Extract Entities & Relationships)**: Từ các đoạn tài liệu đã được phân đoạn, hệ thống sẽ nhận diện và trích xuất các thực thể (ví dụ: tên người, địa điểm, tổ chức, khái niệm) và các mối quan hệ giữa chúng.
4. **Tạo Mô tả (Generate Descriptions)**: Sau khi các thực thể và mối quan hệ được trích xuất, hệ thống sẽ tạo ra các mô tả chi tiết cho từng thực thể. Các mô tả này cung cấp thêm ngữ cảnh và thông tin về thực thể.
5. **Lưu trữ vào Kho khóa-giá trị (Store in Key-Value Store)**: Các thực thể, mối quan hệ và mô tả được lưu trữ trong một kho khóa-giá trị. Đây là một cơ sở dữ liệu đơn giản nhưng hiệu quả để lưu trữ và truy xuất dữ liệu nhanh chóng.
6. **Khử trùng lặp Thực thể (Deduplicate Entities)**: Trong quá trình trích xuất, có thể có các thực thể trùng lặp hoặc các biến thể của cùng một thực thể. Bước này giúp loại bỏ các bản sao và chuẩn hóa các thực thể để đảm bảo tính nhất quán.
7. **Trích xuất Từ khóa Cấp cao (Extract High-Level Keywords)**: Từ các thực thể và mối quan hệ đã được xử lý, hệ thống sẽ trích xuất các từ khóa cấp cao, đại diện cho các chủ đề hoặc khái niệm chính trong tài liệu.
8. **Xây dựng Đồ thị Tri thức (Build Knowledge Graph)**: Đây là bước cuối cùng và quan trọng nhất, nơi tất cả các thông tin đã được xử lý (thực thể, mối quan hệ, mô tả, từ khóa) được tổng hợp để xây dựng Đồ thị Tri thức. Đồ thị này bao gồm:
    - **Các nút Thực thể (Entity Nodes)**: Đại diện cho các thực thể đã được trích xuất.
    - **Các nút Khái niệm (Concept Nodes)**: Đại diện cho các khái niệm hoặc từ khóa cấp cao.

- **Các cạnh (Edges)**: Đại diện cho các mối quan hệ giữa các nút thực thể và nút khái niệm, thể hiện cách chúng liên kết với nhau.

CHƯƠNG 2: PHÂN TÍCH CHI TIẾT CÁC THÀNH PHẦN VÀ QUY TRÌNH HOẠT ĐỘNG CỦA LIGHTRAG

2.1. Kiến trúc tổng quan của LightRAG

LightRAG được thiết kế với một kiến trúc module hóa cao, tập trung vào việc tích hợp chặt chẽ giữa các thành phần xử lý ngôn ngữ tự nhiên truyền thống và kiến trúc tri thức đồ thị. Cấu trúc thư mục chính của LightRAG phản ánh rõ ràng sự phân tách các chức năng **cốt** lõi:

```
lightrag/
├── lightrag.py            # Core engine
├── storage.py             # Storage abstractions
├── llm.py                 # LLM integrations
├── prompt.py              # Vietnamese prompts
├── operate.py             # Graph operations
├── utils.py               # Utilities
├── base.py                # Base classes
└── context_support.py     # Context handling
```

Hình 5. Cấu trúc thư mục của LightRAG

Trong đó, LightRAG Class (được định nghĩa trong lightrag.py) đóng vai trò là trung tâm điều phối của toàn bộ hệ thống. Lớp này quản lý và điều khiển luồng dữ liệu giữa các thành phần chính như:

- **Vector Databases:** Để lưu trữ các vector nhúng (embeddings) cho các thực thể (entities), mối quan hệ (relationships) và các đoạn văn bản (chunks).
- **Knowledge Graph:** Để quản lý và biểu diễn các mối quan hệ phức tạp giữa các thực thể được trích xuất từ dữ liệu.

- **Document Processing:** Xử lý và chia nhỏ tài liệu nguồn thành các chunks, đồng thời trích xuất thực thể và mối quan hệ.
- **LLM Integration:** Tích hợp với các mô hình ngôn ngữ lớn để sinh câu trả lời dựa trên ngữ cảnh được truy xuất.

2.2. Kiến trúc lưu trữ đa tầng

LightRAG sử dụng kiến trúc lưu trữ trừu tượng (Storage Abstraction) linh hoạt, cho phép người dùng dễ dàng thay đổi các backend lưu trữ tùy theo nhu cầu và quy mô ứng dụng. Điều này giúp hệ thống có thể mở rộng và tương thích với nhiều công nghệ cơ sở dữ liệu khác nhau.

2.2.1. Storage Abstraction

Kiến trúc này cung cấp các tùy chọn lưu trữ riêng biệt cho từng loại dữ liệu:

- **Vector Storage Options:**
    - "NanoVectorDBStorage": Một tùy chọn nhẹ, thường dùng cho các thử nghiệm hoặc ứng dụng quy mô nhỏ.
    - "ChromaVectorDBStorage": Tích hợp với ChromaDB, một cơ sở dữ liệu vector mã nguồn mở phổ biến.
    - "MilvusVectorDBStorge": Tích hợp với Milvus, một hệ thống cơ sở dữ liệu vector quy mô lớn.
- **Graph Storage Options:**
    - "NetworkXStorage": Sử dụng thư viện NetworkX cho việc lưu trữ và thao tác đồ thị trong bộ nhớ, phù hợp cho các đồ thị có kích thước vừa phải.
    - "Neo4JStorage": Tích hợp với Neo4j, một cơ sở dữ liệu đồ thị hàng đầu, lý tưởng cho các đồ thị tri thức phức tạp và quy mô lớn.

    - "TiDBGraphStorage": Tích hợp với TiDB Graph, cung cấp khả năng lưu trữ đồ thị phân tán.

- **KV Storage Options:**
    - "JsonKVStorage": Lưu trữ dữ liệu khóa-giá trị dưới dạng JSON, đơn giản và dễ sử dụng.
    - "MongoKVStorage": Tích hợp với MongoDB, một cơ sở dữ liệu NoSQL phổ biến cho việc lưu trữ dữ liệu phi cấu trúc.
    - "OracleKVStorage": Tích hợp với Oracle NoSQL Database, cung cấp khả năng lưu trữ khóa-giá trị đáng tin cậy.

2.2.2. Document Status Tracking

Để quản lý hiệu quả quá trình xử lý tài liệu, LightRAG triển khai hệ thống theo dõi trạng thái. Điều này đặc biệt hữu ích trong các pipeline xử lý dữ liệu lớn, giúp nhà phát triển dễ dàng kiểm soát và gỡ lỗi:

- **PENDING:** Tài liệu đang chờ được xử lý.
- **PROCESSING:** Tài liệu đang trong quá trình xử lý (ví dụ: chia chunk, trích xuất thực thể).
- **PROCESSED:** Tài liệu đã được xử lý hoàn tất và sẵn sàng để truy vấn.
- **FAILED:** Xảy ra lỗi trong quá trình xử lý tài liệu.

2.3. Xử lý ngôn ngữ tiếng Việt

Một trong những điểm mạnh cốt lõi của LightRAG là sự tối ưu hóa chuyên biệt cho ngôn ngữ tiếng Việt, điều này được thể hiện rõ qua việc định nghĩa các loại thực thể và các wrapper LLM tùy chỉnh.

2.3.1. Entity Types được tối ưu cho lịch sử Việt Nam

Trong tệp prompt.py, LightRAG định nghĩa một tập hợp phong phú gồm hơn 25 loại thực thể (ENTITY_TYPES) được thiết kế đặc biệt để phù hợp với ngữ cảnh và đặc thù của lịch sử Việt Nam. Điều này giúp hệ thống có khả năng nhận diện và phân loại các thông tin quan trọng một cách chính xác hơn trong dữ liệu lịch sử.

Ví dụ về các loại thực thể:

```
ENTITY_TYPES = [
    "Nhân vật",
    "Địa điểm",
    "Sự kiện",
    "Triều đại",
    "Tổ chức",
    "Thời kỳ",
    "Văn bản/Hiệp ước/Luật",
    "Giai cấp xã hội",
    "Đơn vị quân sự",
    "Xung đột/Chiến tranh",
    # ... và nhiều loại khác được tối ưu cho lịch sử Việt Nam
]
```

Hình 6. Các loại thực thể phù hợp với lĩnh vực Lịch sử

Việc định nghĩa các loại thực thể này là nền tảng cho quá trình trích xuất thông tin và xây dựng đồ thị tri thức, giúp hệ thống hiểu sâu hơn về ngữ nghĩa của văn bản tiếng Việt.

2.3.2. Vietnamese LLM Wrapper

Tệp llm.py của LightRAG cung cấp các wrapper chuyên biệt cho các Mô hình Ngôn ngữ Lớn (LLM) nhằm đảm bảo rằng phản hồi được sinh ra luôn bằng tiếng Việt và với văn phong tự nhiên, trôi chảy của tiếng Việt. Điều này khắc phục hạn chế của một số LLM tổng quát có thể không tối ưu hoàn toàn cho ngôn ngữ bản địa.

Ví dụ về wrapper cho Gemini:

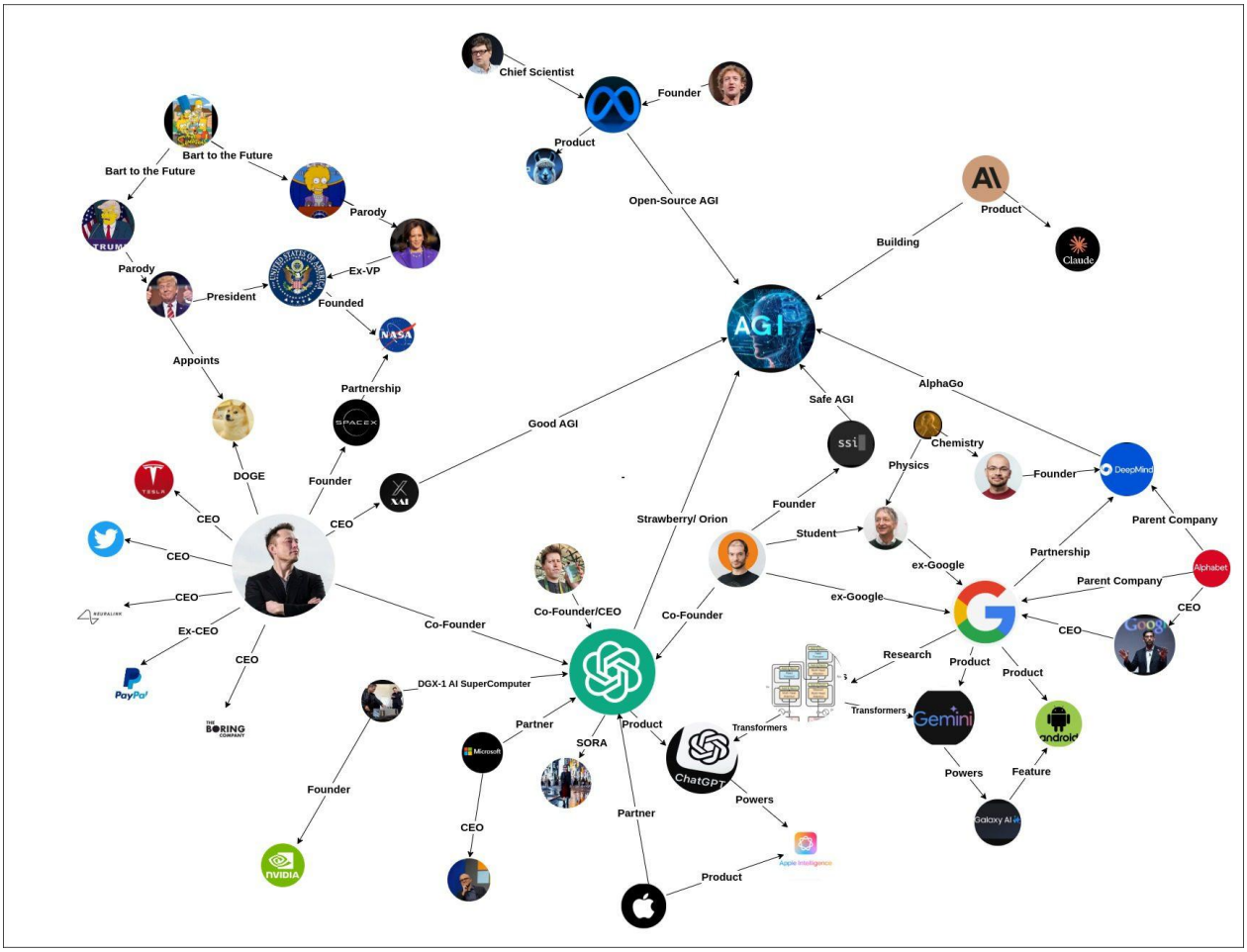
```
async def gemini_complete_vietnamese(prompt, system_prompt=None, **kwargs):
    vi_instruction = """
    Bạn PHẢI trả lời HOÀN TOÀN bằng tiếng Việt.
    Luôn trả lời với văn phong tự nhiên của tiếng Việt.
    """
    # Logic để gọi API của Gemini với prompt đã được thêm hướng dẫn tiếng Việt
    # ...
```

Hình 7. Wrapper Tiếng Việt cho Gemini

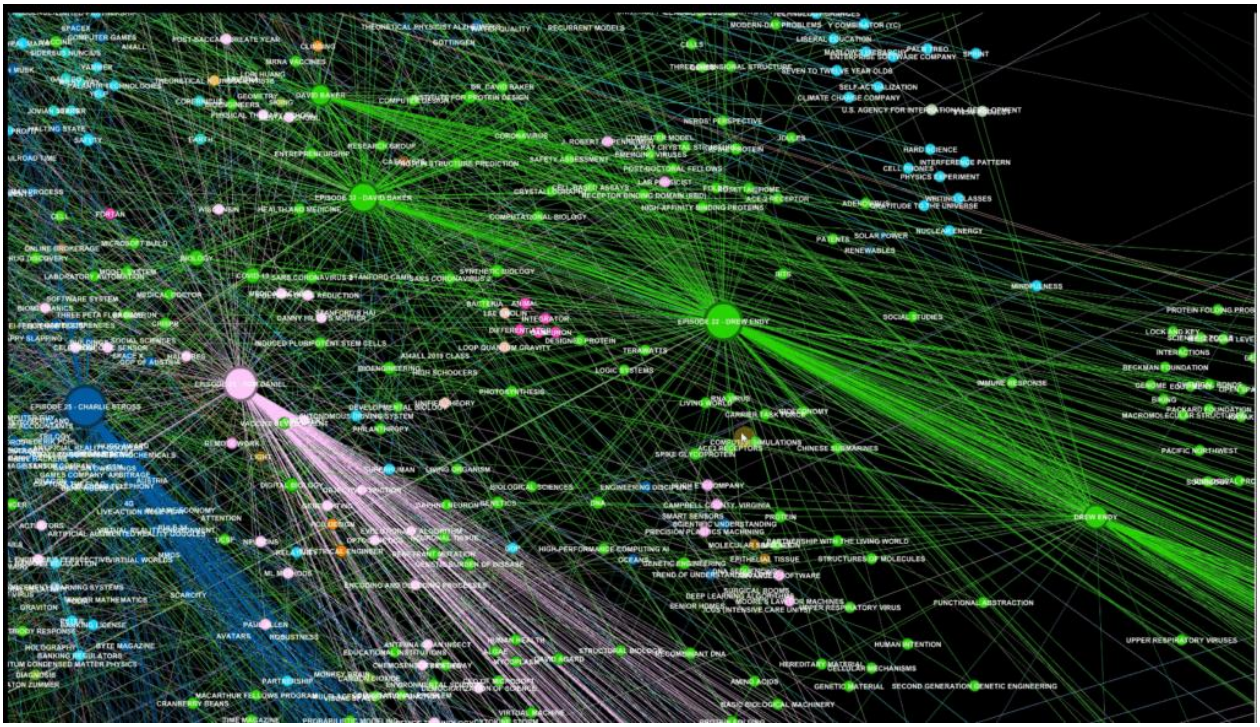
Wrapper này chèn các hướng dẫn cụ thể vào prompt gửi đến LLM, đảm bảo chất lượng và tính bản địa của câu trả lời.

2.4. Knowledge Graph Operations

Khả năng xây dựng và thao tác trên đồ thị tri thức là một tính năng nổi bật của LightRAG, cho phép hệ thống nắm bắt và truy vấn các mối quan hệ phức tạp giữa các thực thể.



Hình 8. Ví dụ về Đồ thị tri thức

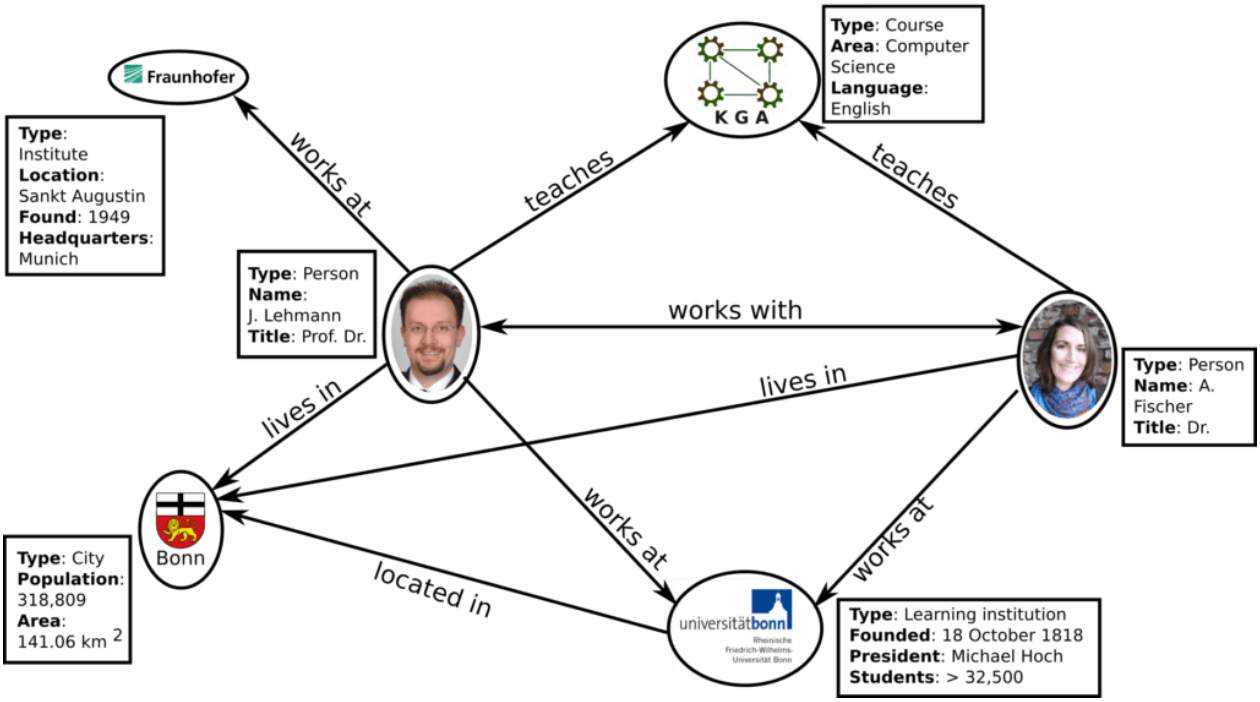


Hình 9. Trực quan hóa đồ thị tri thức

2.4.1. Entity và Relationship Extraction

Hệ thống tự động trích xuất thực thể và mối quan hệ từ văn bản tiếng Việt thông qua một pipeline gồm nhiều bước:

- **Entity Identification:** Xác định các thực thể trong văn bản và phân loại chúng theo hơn 25 loại thực thể đã định nghĩa sẵn (ví dụ: "Nhân vật", "Sự kiện", "Địa điểm").
- **Relationship Mapping:** Ánh xạ các mối quan hệ giữa các thực thể đã xác định, đồng thời gán một độ mạnh (strength) cho mỗi mối quan hệ (từ 1-10) để biểu thị mức độ liên quan hoặc tầm quan trọng.
- **Knowledge Graph Construction:** Dựa trên các thực thể và mối quan hệ đã trích xuất, hệ thống xây dựng một đồ thị tri thức, nơi các thực thể là các nút (nodes) và mối quan hệ là các cạnh (edges).



Hình 10. Ví dụ mô tả các nút và các cạnh trong đồ thị tri thức

2.4.2. Graph Storage và Query

Tệp operate.py chịu trách nhiệm xử lý các thao tác trên đồ thị tri thức, tận dụng các tùy chọn Graph Storage đã được cấu hình:

- **Tìm kiếm thực thể liên quan:** Cho phép tìm kiếm các thực thể có liên quan đến một thực thể hoặc một truy vấn cụ thể trong đồ thị.
- **Tính toán độ quan trọng của mối quan hệ:** Đánh giá mức độ ảnh hưởng hoặc liên kết giữa các thực thể thông qua các mối quan hệ của chúng.
- **Tổng hợp thông tin từ multiple sources:** Khả năng tổng hợp thông tin từ nhiều nguồn khác nhau và tích hợp chúng vào đồ thị tri thức để tạo ra một cái nhìn toàn diện.

2.5. Query Modes

LightRAG hỗ trợ nhiều chế độ truy vấn khác nhau, cho phép người dùng lựa chọn chiến lược phù hợp nhất với loại câu hỏi và yêu cầu về ngữ cảnh:

- **Local Mode:** Tìm kiếm thông tin cục bộ xung quanh các thực thể cụ thể. Chế độ này hữu ích khi người dùng muốn biết chi tiết về một thực thể đã biết.
- **Global Mode:** Tìm kiếm toàn cục trên toàn bộ đồ thị tri thức. Chế độ này phù hợp cho các câu hỏi tổng quát hoặc khi cần khám phá các mối quan hệ rộng lớn.
- **Hybrid Mode:** Kết hợp tìm kiếm cục bộ và toàn cục để cung cấp một cái nhìn cân bằng giữa chi tiết và tổng quan.
- **Naive Mode:** Chế độ RAG truyền thống, chỉ dựa vào tìm kiếm tương đồng vector (vector similarity) trên các chunks văn bản, không sử dụng đồ thị tri thức.
- **Mix Mode:** Kết hợp sức mạnh của đồ thị tri thức và tìm kiếm vector để cung cấp câu trả lời toàn diện và chính xác nhất, tận dụng cả ngữ nghĩa văn bản và cấu trúc mối quan hệ.

2.6. Context Support

Tệp context_support.py cung cấp các chức năng quan trọng để quản lý và tăng cường ngữ cảnh cho quá trình sinh câu trả lời của LLM, đặc biệt hữu ích trong các cuộc hội thoại đa lượt.

- enhance_prompt_with_context(prompt: str, context: str) -> str: Hàm này chịu trách nhiệm tăng cường prompt gốc bằng cách chèn ngữ cảnh đã được truy xuất (từ vector store hoặc knowledge graph) vào một vị trí thích hợp trong prompt.
- extract_relevant_context(history: List[Dict], query: str) -> List[Dict]: Hàm này giúp trích xuất ngữ cảnh liên quan từ lịch sử hội thoại trước đó, đảm bảo LLM có thể ghi nhớ và duy trì ngữ cảnh xuyên suốt cuộc trò chuyện.

2.7. Document Management

LightRAG cung cấp các API mạnh mẽ để quản lý vòng đời của tài liệu trong hệ thống, từ việc chèn dữ liệu mới đến khả năng xóa bỏ.

2.7.1. Insertion Pipeline

Hàm ainsert(self, string_or_strings) là điểm vào chính cho việc chèn tài liệu mới vào LightRAG. Pipeline chèn này tự động hóa nhiều bước phức tạp:

1. **Document chunking:** Chia nhỏ tài liệu thành các đoạn (chunks) phù hợp.
2. **Entity extraction:** Trích xuất các thực thể từ các chunks văn bản.
3. **Relationship identification:** Xác định và ánh xạ các mối quan hệ giữa các thực thể.
4. **Vector storage:** Lưu trữ các vector nhúng của chunks và các thực thể vào Vector Database.
5. **Graph construction:** Xây dựng hoặc cập nhật đồ thị tri thức với các thực thể và mối quan hệ mới.

2.7.2. Deletion Capabilities

Hệ thống hỗ trợ các chức năng xóa linh hoạt để quản lý kho tri thức:

- adelete_by_doc_id(): Xóa tài liệu dựa trên ID duy nhất của tài liệu.
- adelete_by_entity(): Xóa tất cả thông tin (chunks, thực thể, mối quan hệ) liên quan đến một thực thể cụ thể.

2.8. Tính năng nâng cao

LightRAG tích hợp một số tính năng nâng cao để tối ưu hóa hiệu năng và mở rộng khả năng tùy chỉnh.

2.8.1. Custom Knowledge Graph Import

Hàm *ainsert_custom_kg()* cho phép người dùng nhập một đồ thị tri thức có sẵn vào LightRAG. Điều này rất hữu ích khi đã có dữ liệu KG được xây dựng từ trước hoặc muốn tích hợp KG từ các nguồn bên ngoài.

```
custom_kg = {
    "entities": [...],    # Danh sách các thực thể
    "relationships": [...], # Danh sách các mối quan hệ
    "chunks": [...]       # Các chunks văn bản liên quan
}
# ainsert_custom_kg(custom_kg)
```

Hình 11. Custom Knowledge Graph Import

2.8.2. Batch Processing

LightRAG hỗ trợ xử lý tài liệu theo batch để tối ưu hóa hiệu năng, đặc biệt khi làm việc với lượng lớn dữ liệu. Tham số insert_batch_size có thể được cấu hình để điều chỉnh kích thước batch.

```
# batch_size = self.addon_params.get("insert_batch_size", 10)
# Logic xử lý theo batch
```

Hình 12. Batch Processing

2.8.3. Entity Information Retrieval

Hệ thống cung cấp các hàm tiện ích để truy xuất thông tin chi tiết về thực thể và mối quan hệ từ đồ thị tri thức:

- *async def get_entity_info(self, entity_name: str, include_vector_data: bool = False):* Lấy thông tin về một thực thể cụ thể, có thể bao gồm cả dữ liệu vector nhúng liên quan.

- *async def get_relation_info(self, src_entity: str, tgt_entity: str, include_vector_data: bool = False):* Lấy thông tin về mối quan hệ giữa hai thực thể nguồn và đích, bao gồm dữ liệu vector nhúng.

CHƯƠNG 3: MINH HỌA ỨNG DỤNG LIGHTRAG TRONG XÂY DỰNG CHATBOT LỊCH SỬ

3.1. Bối cảnh bài toán ứng dụng (Chatbot hỏi đáp Lịch sử)

Môn Lịch sử đóng vai trò quan trọng trong giáo dục phổ thông, giúp học sinh hiểu về quá khứ, cội nguồn dân tộc và các sự kiện thế giới. Tuy nhiên, việc học Lịch sử theo cách truyền thống thường gặp phải những thách thức như lượng kiến thức lớn, khó nhớ, và thiếu tính tương tác. Một chatbot hỏi đáp Lịch sử thông minh có thể:

- Cung cấp kênh tra cứu thông tin nhanh chóng, tiện lợi.
- Giải đáp thắc mắc cụ thể về sự kiện, nhân vật, khái niệm lịch sử.
- Tạo môi trường học tập tương tác, khuyến khích học sinh chủ động.
- Giảm tải cho giáo viên với các câu hỏi lặp lại.

Chatbot sẽ sử dụng **Sách giáo khoa Lịch sử lớp 10**, Chương trình giáo dục 2006 làm nguồn tri thức chính thống, quen thuộc với học sinh. LightRAG đặc biệt phù hợp cho chatbot lịch sử Việt Nam do các *ENTITY_TYPES* được tối ưu và khả năng xử lý tri thức đồ thị, giúp hệ thống hiểu sâu sắc hơn về các mối quan hệ và ngữ cảnh trong dữ liệu lịch sử.

Mục tiêu của ứng dụng:

- Xử lý và lập chỉ mục nội dung sách giáo khoa, bao gồm cả việc trích xuất thực thể và mối quan hệ.
- Cho phép học sinh đặt câu hỏi bằng ngôn ngữ tự nhiên.
- Truy xuất thông tin liên quan từ sách giáo khoa, tận dụng cả tìm kiếm vector và đồ thị tri thức.
- Sử dụng LLM để tổng hợp và sinh câu trả lời chính xác, mạch lạc bằng tiếng Việt.

3.2. Lựa chọn và cấu hình các thành phần LightRAG cho bài toán

Để xây dựng chatbot Lịch sử, chúng ta sẽ cấu hình các thành phần của LightRAG như sau:

3.2.1. Lựa chọn Document Loader, Text Splitter, Embedder phù hợp với dữ liệu Lịch sử

- **Document Loader (Tải tài liệu):**
  - **Nguồn dữ liệu:** Sách giáo khoa Lịch sử lớp 10 (Chương trình 2006), số hóa dưới dạng tệp *.txt*.
  - **Lựa chọn Loader:**
    - Đối với tệp *.txt* hoặc *.docx*: Sử dụng FileSystemLoader kết hợp với thư viện python-docx để đọc nội dung.
    - Đối với tệp PDF: Sử dụng PDFLoader (PyPDFLoader hoặc UnstructuredPDFLoader) để trích xuất văn bản. Cần lưu ý xử lý các cấu trúc phức tạp như bảng biểu, chú thích hình ảnh để đảm bảo tính toàn vẹn của dữ liệu.
  - **Cấu hình:** Chỉ định đường dẫn đến tệp tài liệu nguồn.
- **Text Splitter (Phân tách văn bản):**
  - **Đặc điểm dữ liệu:** Văn bản sách giáo khoa có cấu trúc rõ ràng theo bài, mục, đoạn, với các sự kiện và khái niệm được trình bày độc lập.
  - **Lựa chọn Splitter:** *RecursiveCharacterTextSplitter* là phù hợp, giúp giữ được tính toàn vẹn ngữ nghĩa của các đoạn văn bản.
  - **Cấu hình:**
    - *chunk_size*: 1024 token, đủ để chứa thông tin về một sự kiện hoặc một khía cạnh cụ thể, nhưng không quá dài gây nhiễu.

        - *chunk_overlap*: 128 token, tránh mất mát thông tin quan trọng tại ranh giới giữa các chunks.
        - *separators*: Ưu tiên ["\n\n", "\n", ".", " ", ""] để phân tách theo đoạn, câu, hoặc từ.
    - **Quá trình xử lý bổ sung:** Sau khi chia chunk, LightRAG sẽ tự động sử dụng quá trình xử lý tài liệu để trích xuất các thực thể và mối quan hệ lịch sử từ các chunks này, phục vụ cho việc xây dựng đồ thị tri thức.
- **Embedder (Mô hình tạo Nhúng):**
    - **Yêu cầu:** Mô hình nhúng cần hiểu tốt tiếng Việt và các thuật ngữ chuyên ngành Lịch sử.
    - **Lựa chọn:** Có nhiều lựa chọn phù hợp, điển hình như các mô hình hỗ trợ tiếng Việt như: PhoBERT, viBERT, hoặc các mô hình đa ngôn ngữ như Mistral, XLM-RoBERTa, ... Tuy nhiên, để đơn giản hóa quy trình xây dựng, báo cáo lựa chọn sử dụng mô hình text-embedding-004 của Gemini, thông qua API.
    - **Cấu hình:**
        - Sử dụng lớp ***GeminiEmbeddings*** trong LightRAG.
        - Cung cấp API key và chỉ định tên mô hình text-embedding-004.
    - LightRAG sẽ xử lý việc gọi API Gemini để tạo vector nhúng cho mỗi chunk văn bản, cũng như cho các thực thể và mối quan hệ được trích xuất.

3.2.2. Cấu hình VectorDB và các tham số cho Retriever

- **Lựa chọn VectorDB:**

- **Yêu cầu:** Đối với giai đoạn thử nghiệm ban đầu và triển khai cục bộ, cần một giải pháp nhẹ, dễ sử dụng nhưng vẫn đảm bảo hiệu suất.
- **Lựa chọn:** FAISS là một thư viện mạnh mẽ, hiệu suất cao, phù hợp cho dữ liệu nhỏ đến trung bình. ChromaDB cũng là một lựa chọn thay thế tốt.
- **Quyết định:** Sử dụng FAISS do tính phổ biến và hiệu suất cục bộ tốt.
- **Graph Storage:** Song song với VectorDB, chúng ta sẽ sử dụng một Graph Storage như NetworkXStorage (cho thử nghiệm cục bộ) hoặc Neo4JStorage (nếu cần triển khai quy mô lớn hơn) để lưu trữ đồ thị tri thức được xây dựng.

- **Cấu hình VectorDB (FAISS):**
  - Sử dụng lớp *FAISSVectorStore* trong LightRAG.
  - **Quá trình:**
    - Các vector nhúng từ *GeminiEmbeddings* sẽ được truyền vào phương thức *add_documents()*.
    - FAISS sẽ xây dựng chỉ mục (IndexFlatL2 cho tìm kiếm chính xác hoặc IndexIVFFlat cho tìm kiếm xấp xỉ).
    - Chỉ mục có thể được lưu vào tệp để tái sử dụng mà không cần tạo lại.
- **Cấu hình Retriever:**
  - **Sử dụng:** Retriever của LightRAG sẽ tận dụng cả FAISS để tìm kiếm vector và các thao tác trên đồ thị tri thức.
  - **Tham số:**
    - *search_type*: "similarity" (sử dụng cosine similarity).

        - *k*: 3-4 chunks để cân bằng giữa việc cung cấp đủ thông tin và tránh nhiễu.
        - *query_mode*: Quan trọng là lựa chọn Query Mode phù hợp của LightRAG. Đối với chatbot lịch sử, Mix Mode hoặc Hybrid Mode sẽ được ưu tiên để kết hợp tìm kiếm vector và truy vấn đồ thị, tận dụng tối đa ngữ cảnh từ cả văn bản và mối quan hệ.
        - *score_threshold*: Lọc các chunks có độ tương đồng thấp hơn một ngưỡng nhất định.

3.2.3. Thiết kế prompt cho LLM (Gemini)

- **Lựa chọn LLM:**
    - **Mô hình:** Tương tự như việc lựa chọn mô hình embedding, để đơn giản hóa, báo cáo tiếp tục sử dụng một mô hình đến từ Google Gemini(gemini-2.5-flash) thông qua API.
    - **Cấu hình:**
        - Sử dụng lớp ChatGemini trong LightRAG, được tích hợp với Vietnamese LLM Wrapper.
        - Cung cấp API key và tên mô hình.
        - **Tham số:** *temperature* (0.2-0.5) để đảm bảo câu trả lời nhất quán và ít "ảo giác", *max_output_tokens* để giới hạn độ dài câu trả lời.
- **Mẫu Prompt:**
    - **Bao gồm:**
        - **Chỉ dẫn vai trò:** "Bạn là trợ lý AI chuyên về Lịch sử, dựa trên sách giáo khoa."

- **Ngữ cảnh:** Placeholder {context} cho các chunks văn bản được truy xuất.
- **Thông tin từ đồ thị tri thức:** Có thể thêm placeholder cho thông tin trích xuất từ đồ thị tri thức (ví dụ: {kg_facts}) để làm giàu ngữ cảnh.
- **Câu hỏi:** Placeholder {query} cho câu hỏi của người dùng.
- **Hướng dẫn:** Bám sát ngữ cảnh được cung cấp, trả lời bằng tiếng Việt, xử lý trường hợp không có thông tin liên quan.

- **Ví dụ:**

```
PROMPTS["mix_rag_response"] = """---Role---

You are a professional assistant responsible for answering questions based on
knowledge graph and textual information. Please respond in the same language as
the user's question.

---Goal---

Generate a concise response that summarizes relevant points from the provided
information. If you don't know the answer, just say so. Do not make anything up or
include information where the supporting evidence is not provided.

When handling information with timestamps:

1. Each piece of information (both relationships and content) has a "created_at"
timestamp indicating when we acquired this knowledge

2. When encountering conflicting information, consider both the
content/relationship and the timestamp

3. Don't automatically prefer the most recent information - use judgment based on
the context
```

4. For time-specific queries, prioritize temporal information in the content before considering creation timestamps

---Data Sources---

1. Knowledge Graph Data:

{kg_context}

2. Vector Data:

{vector_context}

---Response Requirements---

- Target format and length: {response_type}

- Use markdown formatting with appropriate section headings

- Aim to keep content around 3 paragraphs for conciseness

- Each paragraph should be under a relevant section heading

- Each section should focus on one main point or aspect of the answer

- Use clear and descriptive section titles that reflect the content

- List up to 5 most important reference sources at the end under "References", clearly indicating whether each source is from Knowledge Graph (KG) or Vector Data (VD)

  Format: [KG/VD] Source content

Add sections and commentary to the response as appropriate for the length and format. If the provided information is insufficient to answer the question, clearly state that you don't know or cannot provide an answer in the same language as the user's question."""

- Việc sử dụng các *ENTITY_TYPES* đã định nghĩa sẵn của LightRAG trong quá trình trích xuất thông tin sẽ giúp làm giàu ngữ cảnh {kg_facts} này. Đồng thời, *gemini_complete_vietnamese* wrapper sẽ đảm bảo phản hồi luôn bằng tiếng Việt.

3.3. Phân tích kết quả thử nghiệm

Sau khi cấu hình và triển khai các thành phần LightRAG, chúng ta tiến hành thử nghiệm với các câu hỏi thực tế để đánh giá khả năng của chatbot Lịch sử.

3.3.1. Minh họa khả năng truy xuất và sinh câu trả lời

- **Ví dụ 1: Câu hỏi về sự kiện và mối quan hệ**
  - **Câu hỏi:** "Trận Bạch Đằng năm 938 có ý nghĩa như thế nào và mối quan hệ giữa Ngô Quyền với quân Nam Hán trong trận này?"
  - **Quá trình:**
    - Câu hỏi được nhúng bằng *text-embedding-004*.
    - Retriever (sử dụng Mix Mode của LightRAG) trả về các chunks văn bản liên quan từ FAISS và thông tin mối quan hệ từ đồ thị tri thức (ví dụ: Ngô Quyền đánh bại Nam Hán, Ngô Quyền chỉ huy trận Bạch Đằng).
    - **Chunks truy xuất:**
      - Chunk 1: "Chiến thắng Bạch Đằng năm 938... khẳng định nền độc lập tự chủ..."
      - Chunk 2: "Trận Bạch Đằng... mở ra thời kỳ độc lập lâu dài..."
      - Chunk 3: "Nghệ thuật quân sự độc đáo trong trận Bạch Đằng do Ngô Quyền chỉ huy..."
    - **Thông tin KG truy xuất**

        - (Ngô Quyền) -[đánh bại]-> (Quân Nam Hán)
        - (Ngô Quyền) -[chỉ huy]-> (Trận Bạch Đằng năm 938)
    - Prompt Builder kết hợp các chunks, thông tin KG và câu hỏi.
    - Gemini (sử dụng *gemini_complete_vietnamese* wrapper) sinh câu trả lời.
  - **Câu trả lời:** "Trận Bạch Đằng năm 938 do Ngô Quyền chỉ huy có ý nghĩa vô cùng quan trọng:
    - Đập tan hoàn toàn ý chí xâm lược của quân Nam Hán, khẳng định nền độc lập tự chủ của dân tộc sau hơn một nghìn năm Bắc thuộc.
    - Mở ra thời kỳ độc lập lâu dài, tạo tiền đề vững chắc cho việc xây dựng các triều đại phong kiến độc lập ở Việt Nam.
    - Ngô Quyền đã đánh bại quân Nam Hán trong trận này, thể hiện tài năng quân sự xuất chúng và vai trò quyết định trong việc giành lại độc lập."
- **Ví dụ 2: Câu hỏi về khái niệm và liên kết**
  - **Câu hỏi:** "Chế độ chiếm hữu nô lệ là gì và nó phổ biến ở những nền văn minh nào?"
  - **Câu trả lời:** "Chế độ chiếm hữu nô lệ là hình thái kinh tế-xã hội trong đó chủ nô sở hữu tư liệu sản xuất và nô lệ, coi nô lệ là 'công cụ biết nói', không có tự do, phải lao động cưỡng bức. Chế độ này phổ biến ở các nền văn minh cổ đại như Hy Lạp và La Mã."
- **Ví dụ 3: Câu hỏi không có thông tin trong nguồn tri thức**
  - **Câu hỏi:** "Vua Hùng thứ 18 có bao nhiêu người con?"

- **Câu trả lời:** "Tôi không tìm thấy thông tin cụ thể về số lượng con của Vua Hùng thứ 18 trong sách giáo khoa Lịch sử lớp 10."

3.3.2. Đánh giá sơ bộ hiệu quả

- **Document Loaders và Text Splitters:**
  - **Hiệu quả:** Dễ dàng xử lý dữ liệu sách giáo khoa. Cần thử nghiệm chunk_size và chunk_overlap để tối ưu hóa việc phân tách.
  - **Thách thức:** Sách giáo khoa có cấu trúc đề mục phức tạp (phần, chương, bài, ...), cần kỹ thuật tiền xử lý nâng cao hơn để trích xuất văn bản chính xác.
- **Embedders (Gemini text-embedding-004):**
  - **Hiệu quả:** Mô hình này cho thấy khả năng hiểu tốt tiếng Việt và các thuật ngữ lịch sử, tích hợp dễ dàng vào LightRAG.
  - **Cân nhắc:** Chi phí API và tốc độ xử lý là yếu tố cần xem xét với lượng dữ liệu lớn.
- **VectorDB (FAISS), Graph Storage và Retriever:**
  - **Hiệu quả:** FAISS mạnh mẽ cho tìm kiếm vector. Việc tích hợp Graph Storage và sử dụng các Query Modes của LightRAG (như Mix Mode) cho phép Retriever truy xuất ngữ cảnh phong phú hơn, bao gồm cả mối quan hệ giữa các thực thể, giúp câu trả lời chính xác và sâu sắc hơn. Khả năng điều chỉnh k (số lượng chunks) và lựa chọn Query Mode phù hợp là rất quan trọng.
  - **Thách thức:** Việc chọn loại chỉ mục FAISS phù hợp và tối ưu hóa hiệu suất truy vấn trên đồ thị tri thức cần được cân nhắc kỹ lưỡng.
- **Prompt Builders và LLM (Gemini):**

- **Hiệu quả:** Việc thiết kế prompt tùy chỉnh kết hợp cả chunks văn bản và thông tin từ đồ thị tri thức đảm bảo LLM nhận được ngữ cảnh đầy đủ. Gemini, với sự hỗ trợ của Vietnamese LLM Wrapper, tổng hợp thông tin tốt và sinh ra câu trả lời mạch lạc, tự nhiên bằng tiếng Việt.
- **Thách thức:** Vẫn cần tinh chỉnh prompt liên tục để giảm thiểu hiện tượng "ảo giác" và đảm bảo LLM luôn bám sát nguồn thông tin cung cấp.

- **Tính dễ sử dụng của LightRAG:**
  - **Ưu điểm:** Kiến trúc module hóa, API rõ ràng và các thành phần được đóng gói sẵn giúp giảm đáng kể công sức phát triển. Khả năng tối ưu hóa cho tiếng Việt và kiến trúc đồ thị tri thức là những lợi thế lớn.
  - **Lưu ý:** Để tận dụng tối đa các tính năng nâng cao (như các Query Modes phức tạp hoặc quản lý đồ thị tri thức), người dùng cần có sự hiểu biết nhất định về kiến trúc RAG và các khái niệm liên quan. Cần có tài liệu và ví dụ minh họa rõ ràng hơn cho các tính năng này.

Ứng dụng LightRAG cho chatbot Lịch sử cho thấy tiềm năng to lớn trong lĩnh vực giáo dục, mang lại một công cụ học tập thông minh và tương tác. Tuy nhiên, để đạt được hiệu quả cao nhất và triển khai thực tế, cần có sự cấu hình cẩn thận, dữ liệu chất lượng, và liên tục tinh chỉnh các thành phần.

3.4. Demo ứng dụng

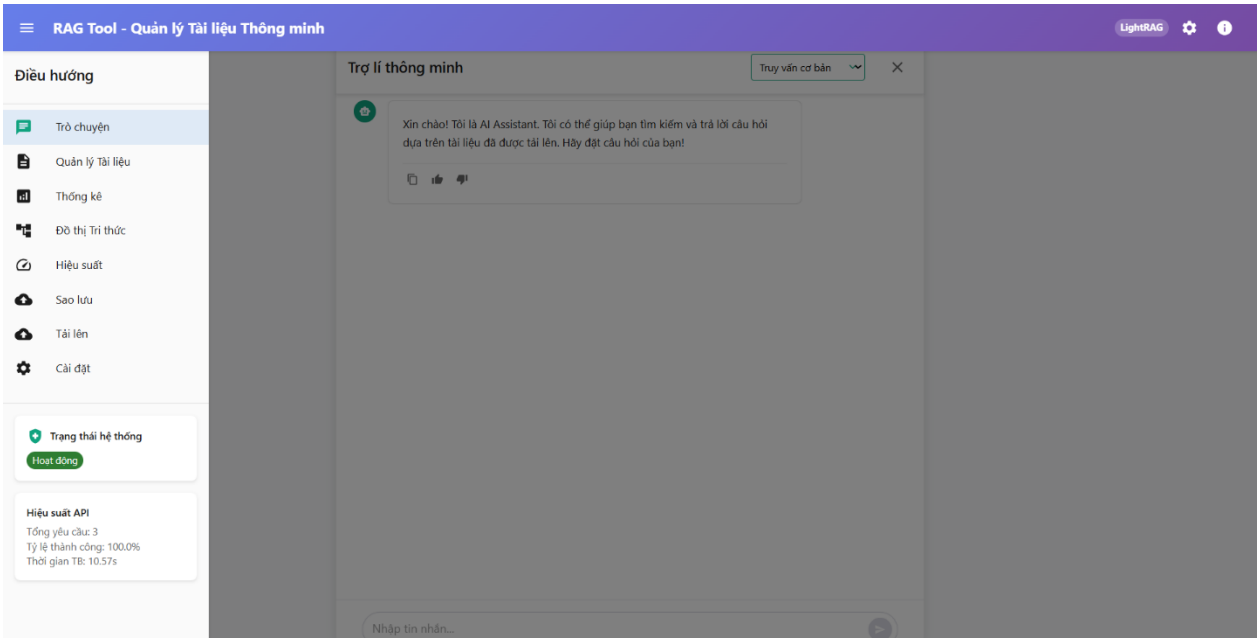
3.4.1. Các tính năng chính của ứng dụng

Ứng dụng được xây dựng với một số tính năng nổi bật như:

- Hỏi đáp các câu hỏi liên quan đến kiến thức Lịch sử dựa trên sách giáo khoa Lịch sử 10.
- Quản lí nguồn tài liệu (Thêm, xóa tài liệu)
- Trực quan hóa đồ thị tri thức

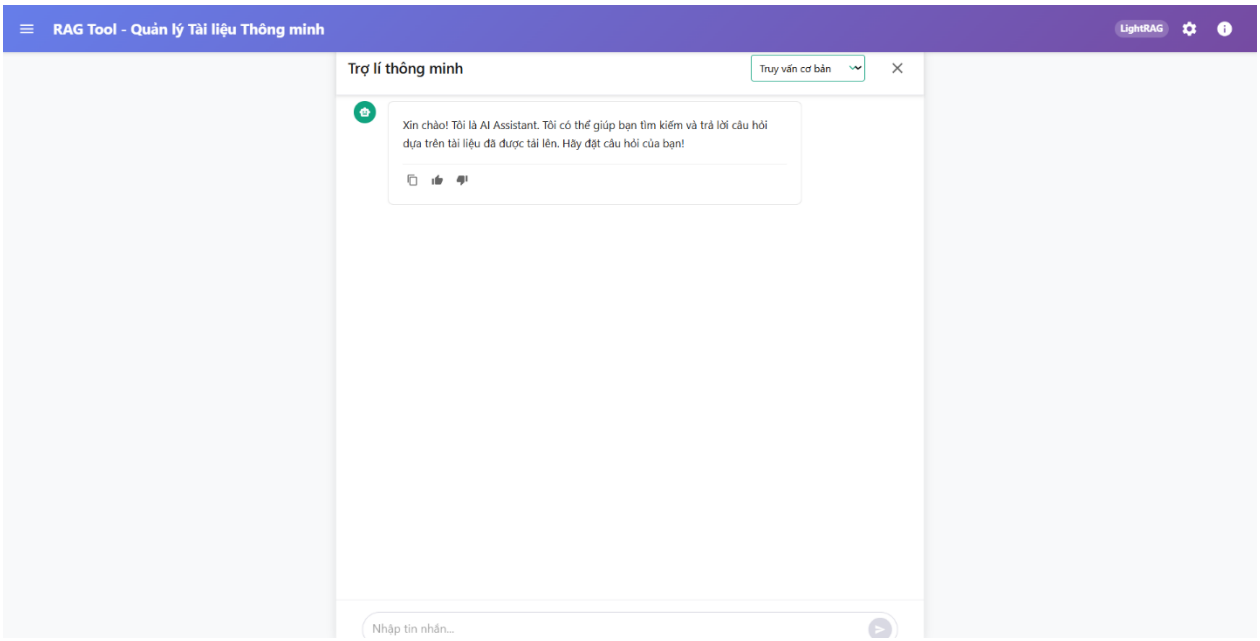
3.4.2. Một số hình ảnh từ ứng dụng được cài đặt

- Giao diện Menu của ứng dụng



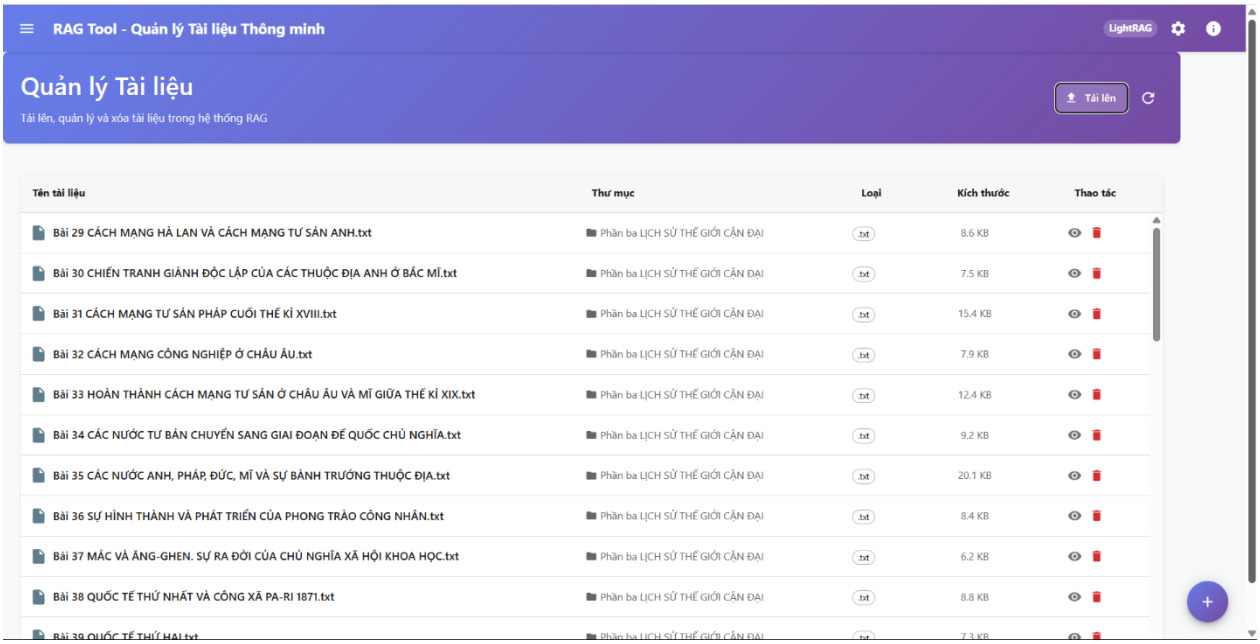
Hình 13. Menu của ứng dụng

- Giao diện chính của Chatbot



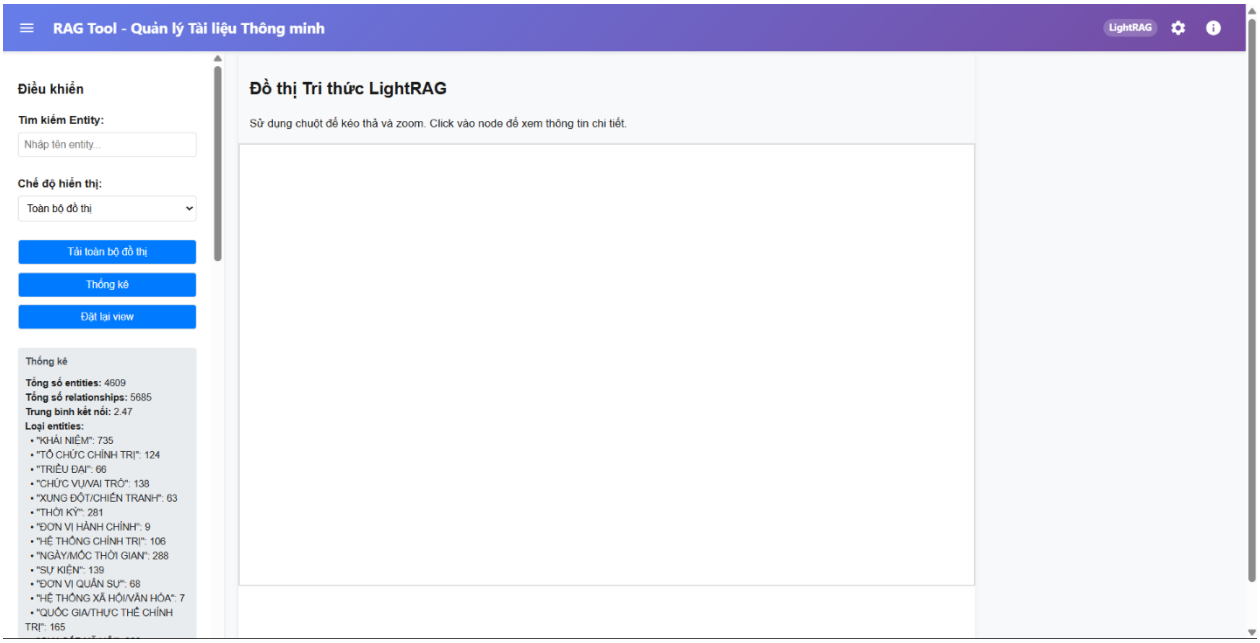
Hình 14. Giao diện Chatbot

- Giao diện quản lí tài liệu. Các chức năng bao gồm: Xem tài liệu, thêm, xóa tài liệu.



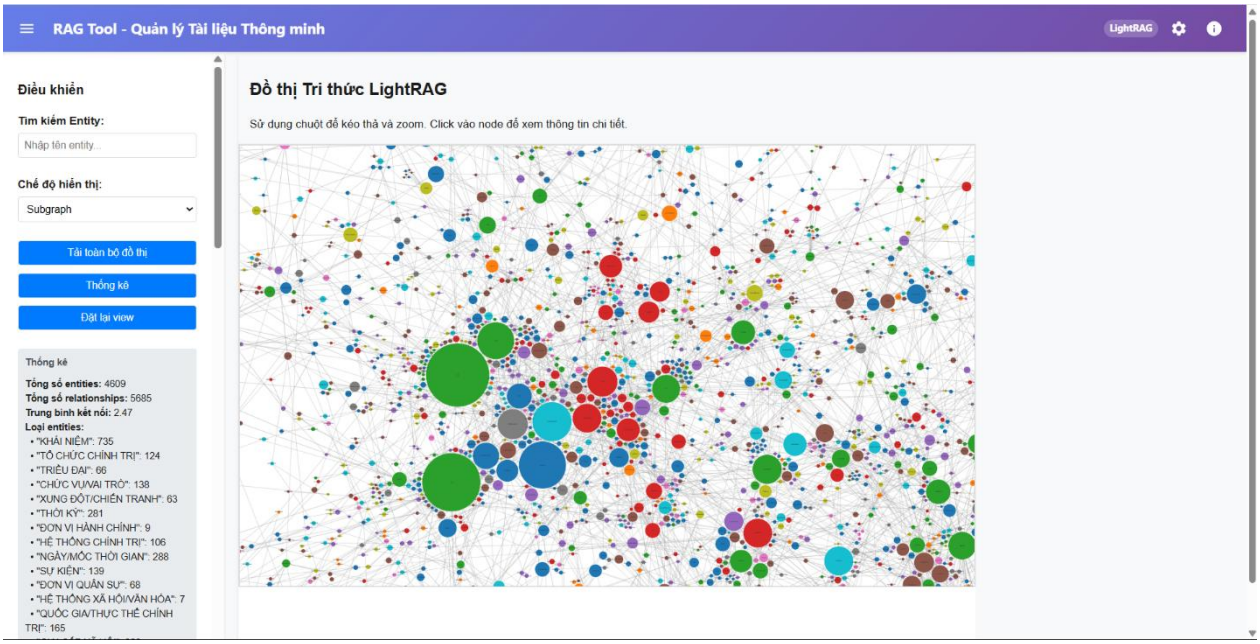
Hình 15. Giao diện quản lí tài liệu

- Giao diện biểu diễn đồ thị tri thức



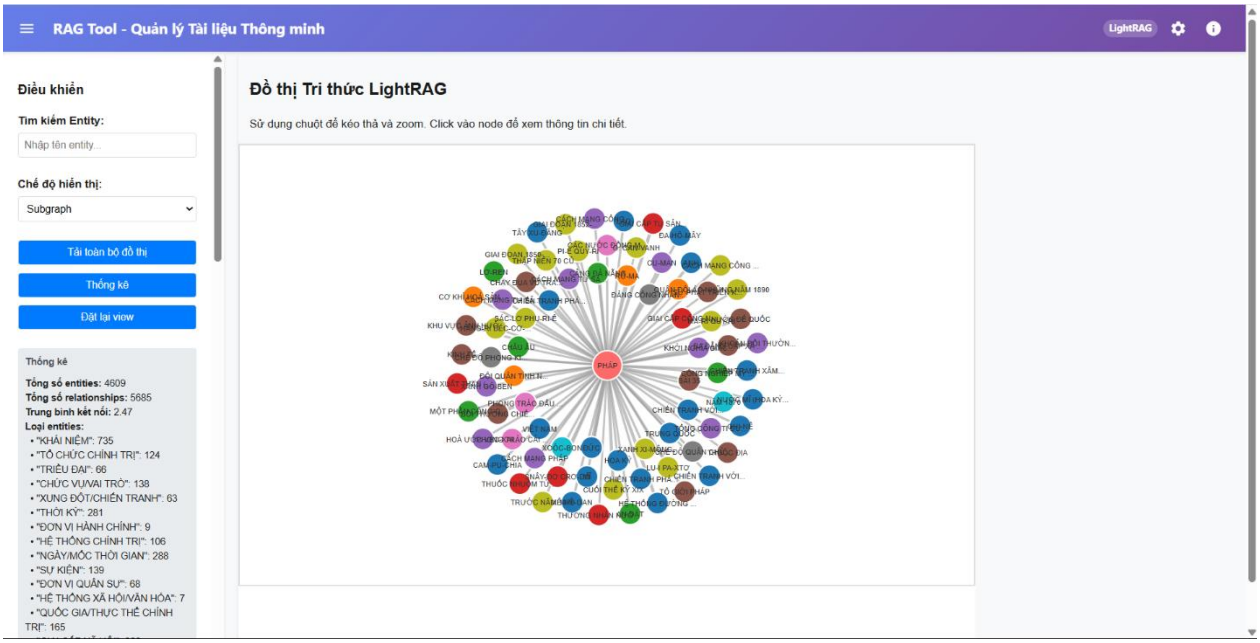
Hình 16. Giao diện biểu diễn đồ thị tri thức

- Toàn bộ thực thể và liên kết đã trích xuất được từ sách giáo khoa Lịch sử 10



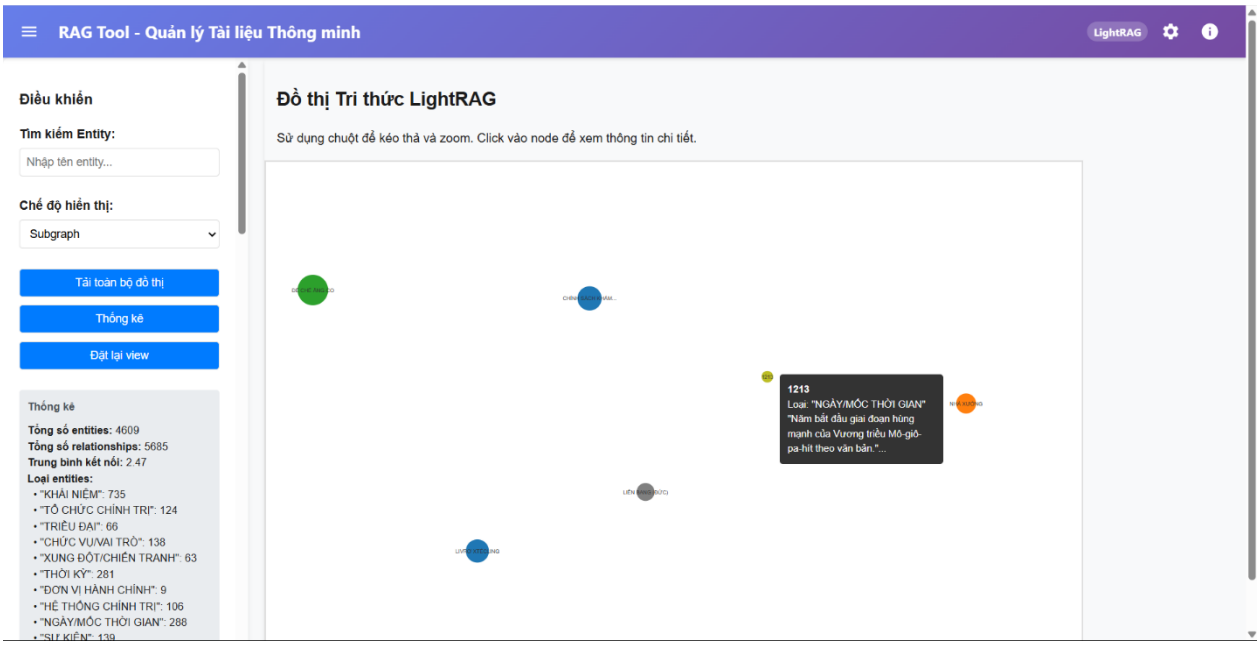
Hình 17. Đồ thị tri thức toàn hệ thống

- Ví dụ về thực thể "Nước Anh" và các mối liên hệ trực tiếp với nó.



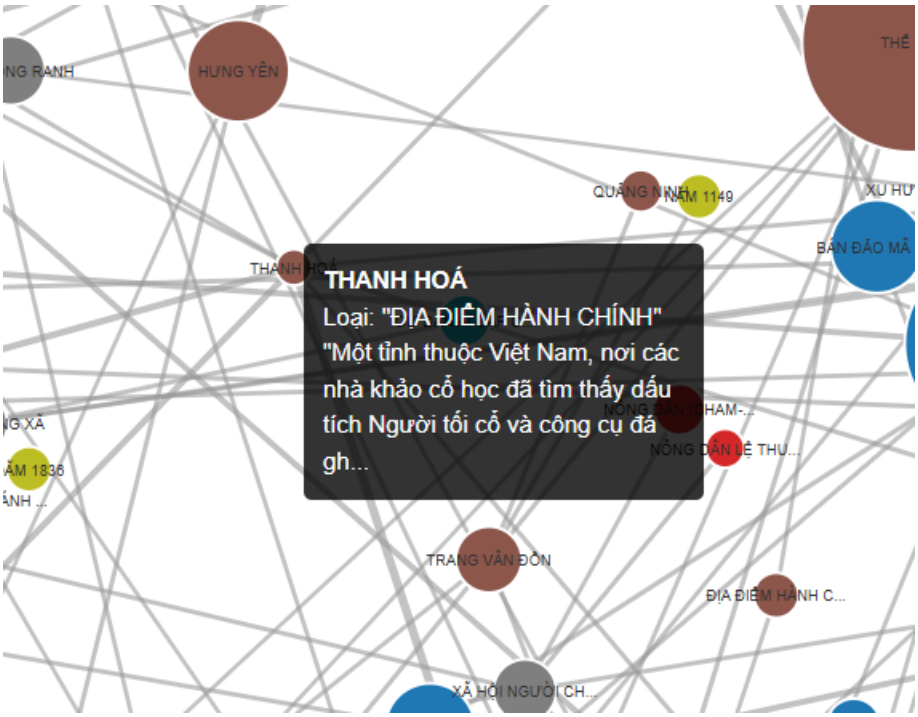
Hình 18. Ví dụ về thực thể trong đồ thị tri thức

- Các thực thể không tồn tại liên kết nào sẽ nằm ở phần biên ngoài của đồ thị.



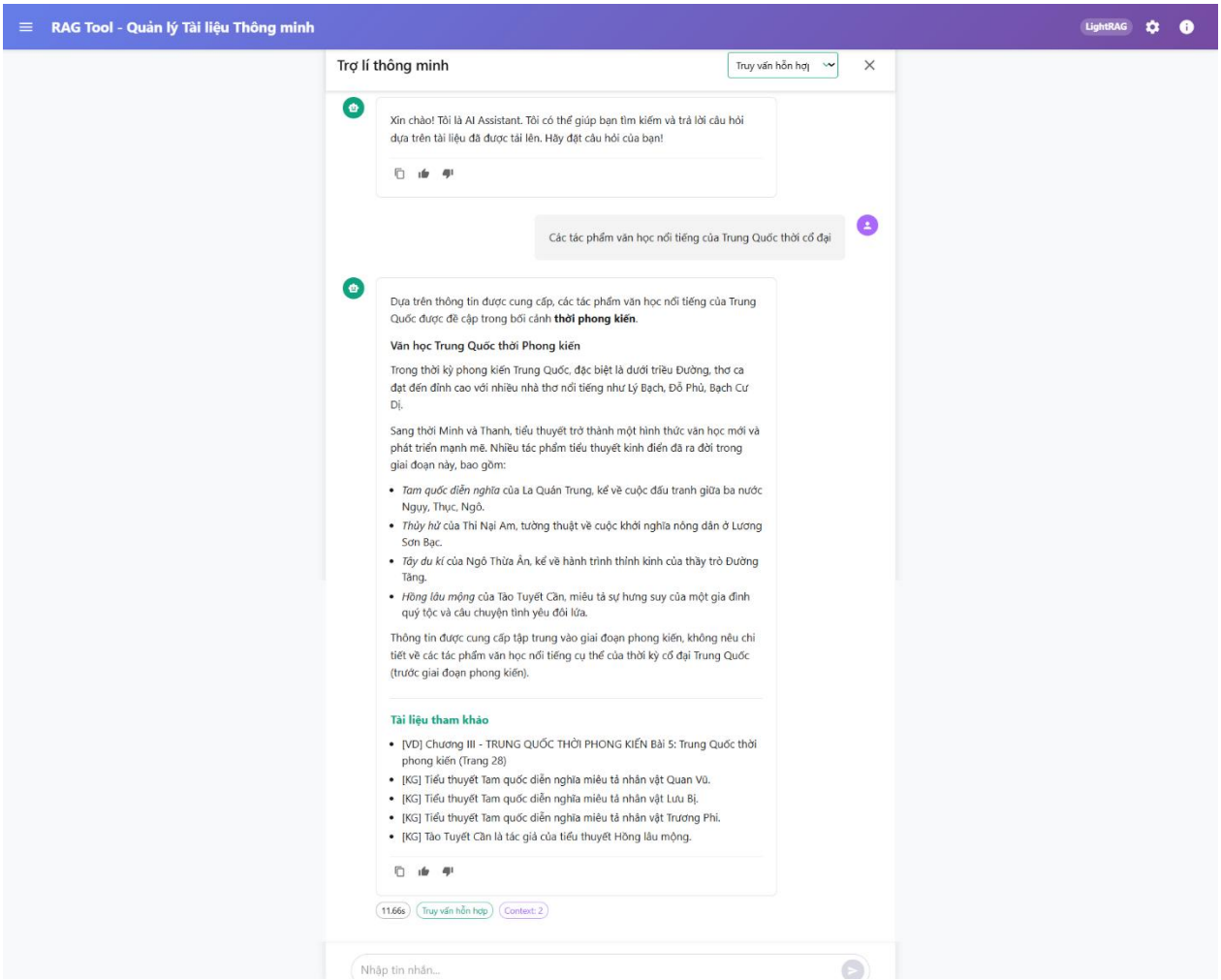
Hình 19. Ví dụ về thực thể trong đồ thị tri thức 2

- Ví dụ về thực thể "Thanh Hóa" và mô tả chi tiết.



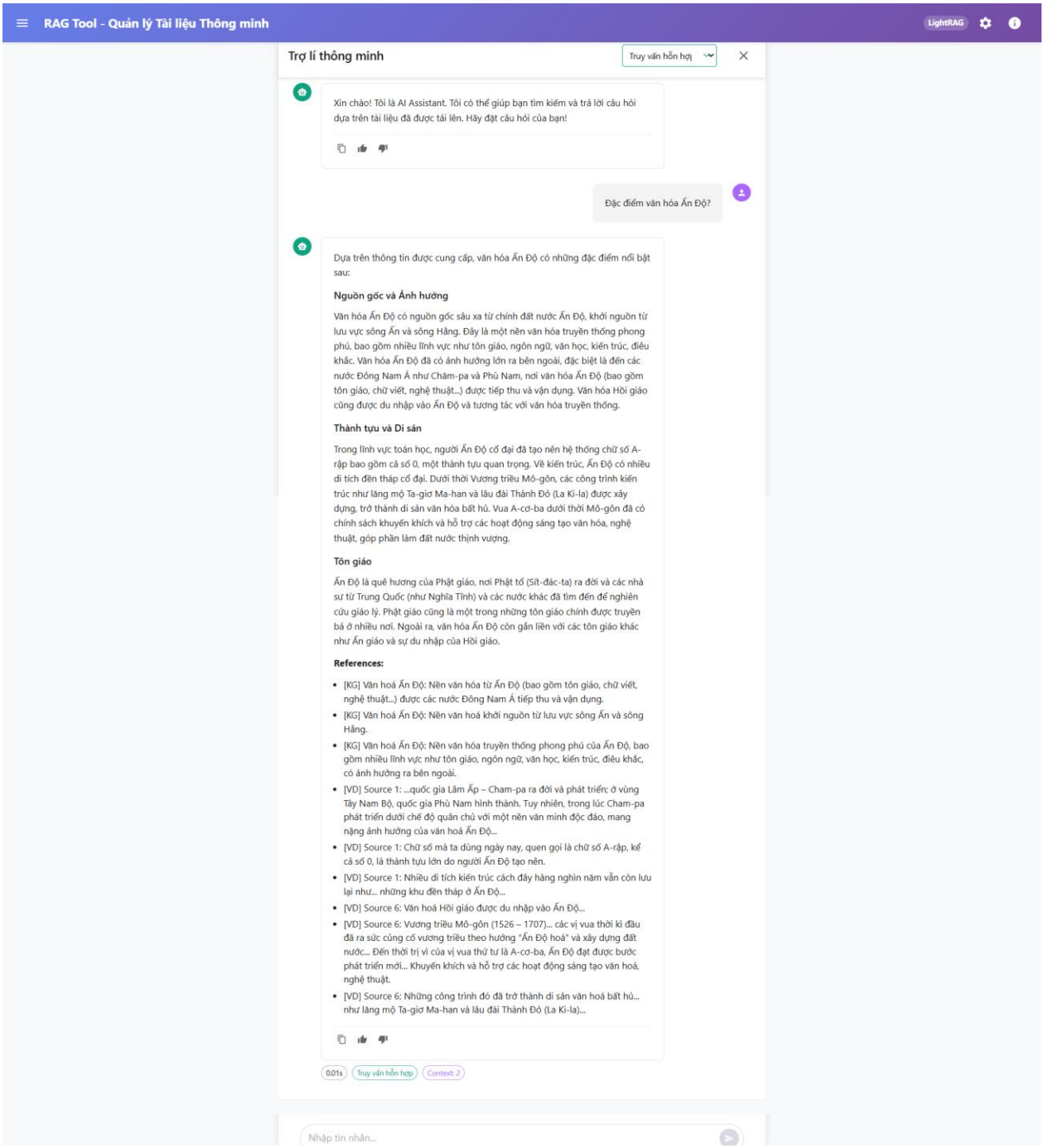
Hình 20. Ví dụ về thực thể trong đồ thị tri thức 3

- Câu hỏi ví dụ mang tính chi tiết



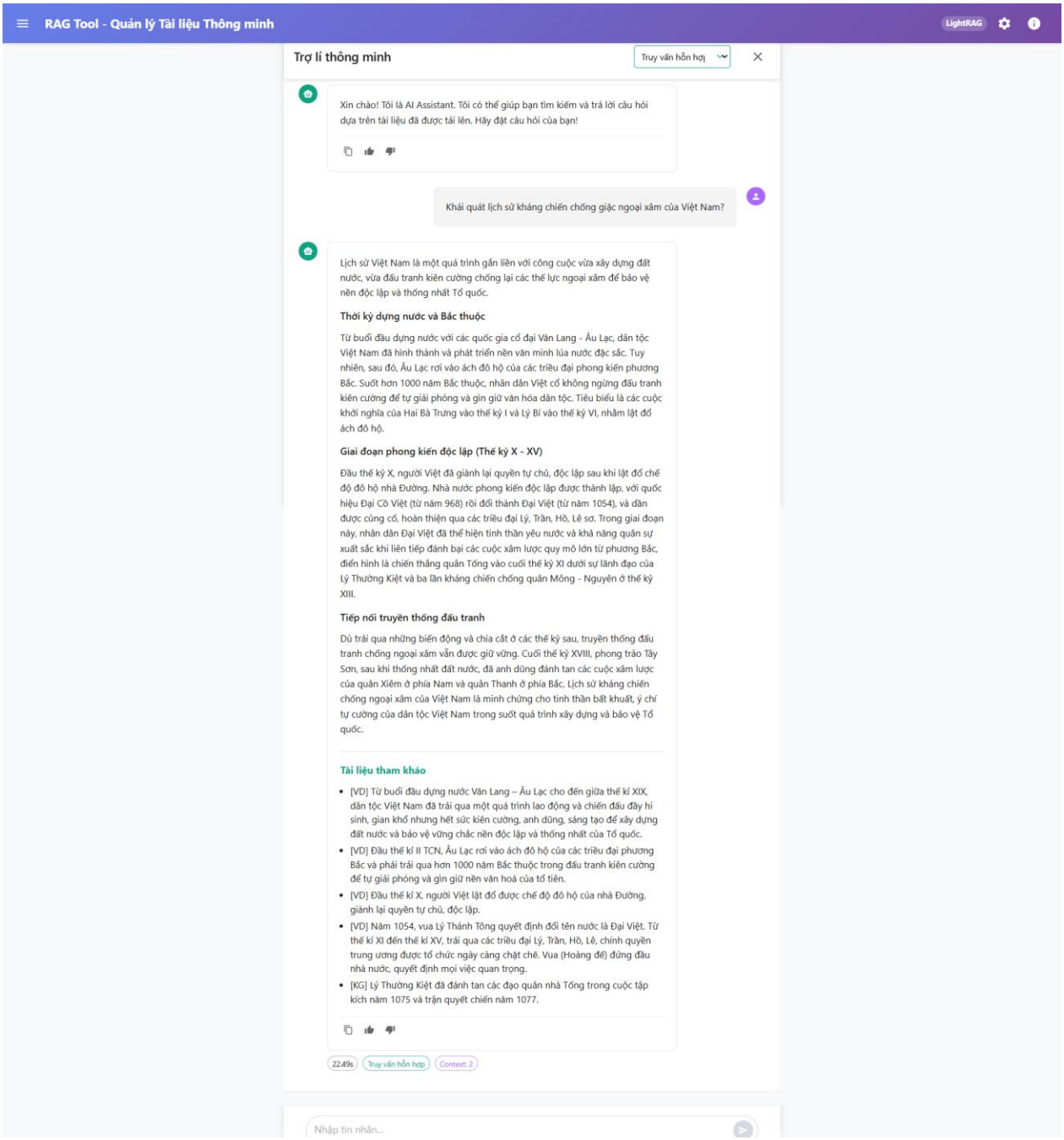
Hình 21. Câu hỏi minh họa

- Câu hỏi ví dụ mang tính tổng quát



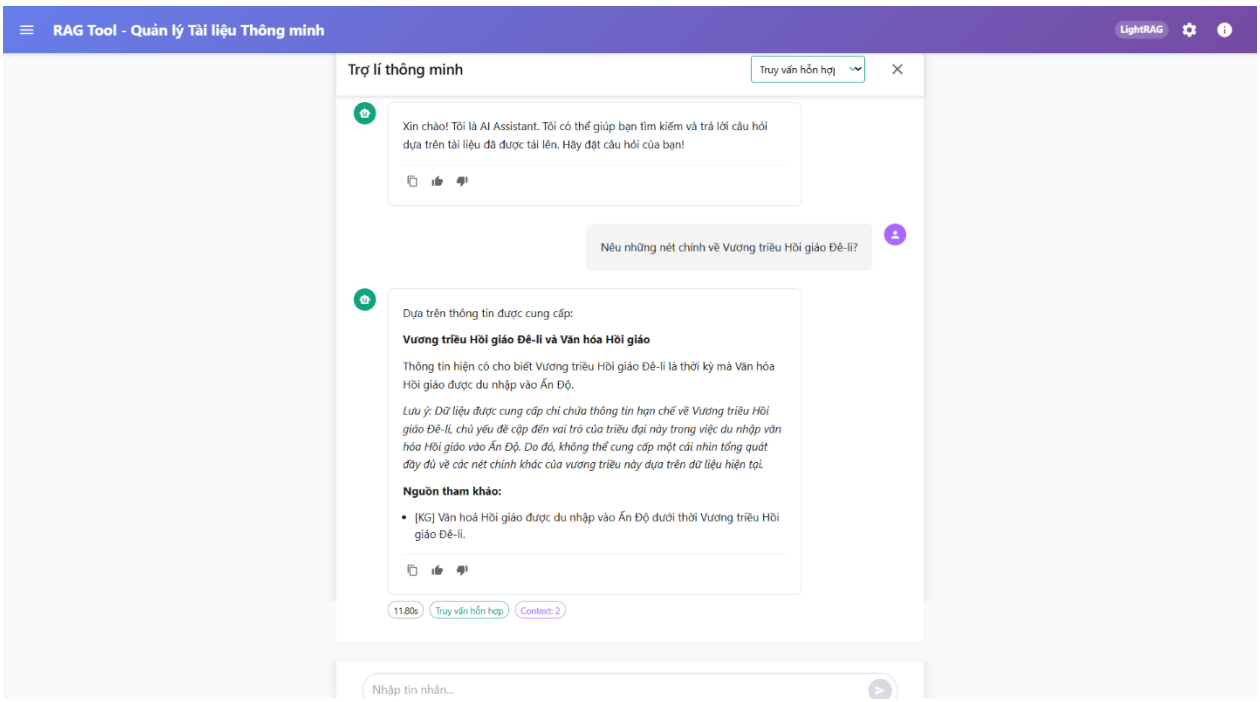
Hình 22. Câu hỏi minh họa 2

- Câu hỏi ví dụ mang tính tổng quát



Hình 23. Câu hỏi minh họa 3

- Câu hỏi ví dụ cho trường hợp không có đáp án hoặc không chắc chắn về đáp án



Hình 24. Câu hỏi minh họa 4

CHƯƠNG 4: KẾT LUẬN VÀ ĐỀ XUẤT

4.1. Tổng kết những phân tích chính về các thành phần của LightRAG

Báo cáo đã đi sâu phân tích các thành phần cốt lõi và quy trình hoạt động của LightRAG, một framework RAG tiên tiến được tối ưu hóa cho tiếng Việt và tích hợp kiến trúc đồ thị tri thức.

4.1.1. Quy trình Xử lý và Lập chỉ mục Dữ liệu (Data Ingestion and Indexing Pipeline)

- **Document Loaders**: Đóng vai trò quan trọng trong việc thu thập dữ liệu từ nhiều nguồn như .txt, .docx, PDF. Tính linh hoạt của LightRAG trong việc hỗ trợ các loader như *FileSystemLoader* và *PDFLoader* giúp xử lý đa dạng định dạng tài liệu.
- **Text Splitters**: Chia nhỏ văn bản (chunking) tối ưu hóa cho nhúng và LLM. *RecursiveCharacterTextSplitter* hiệu quả trong việc giữ ngữ nghĩa, đặc biệt với dữ liệu cấu trúc như sách giáo khoa. Cần thử nghiệm *chunk_size* và *chunk_overlap* để tối ưu.
- **Embedders**: Mô hình nhúng như *text-embedding-004* của Gemini biểu diễn ngữ nghĩa tốt, hỗ trợ tìm kiếm chính xác. Tích hợp dễ dàng trong LightRAG nâng cao chất lượng pipeline.
- **LightRAG nổi bật với khả năng trích xuất thực thể và mối quan hệ (Entity and Relationship Extraction)** từ văn bản tiếng Việt, xây dựng một đồ thị tri thức song song với việc tạo vector nhúng. Các *ENTITY_TYPES* được tối ưu cho lịch sử Việt Nam giúp hệ thống hiểu sâu hơn về nội dung.
- **Vector Stores / VectorDBs**: Các giải pháp như FAISS, ChromaDB cung cấp lưu trữ và truy xuất vector hiệu quả.

- **Kiến trúc lưu trữ đa tầng (Storage Abstraction)** cho phép linh hoạt lựa chọn các backend lưu trữ (Vector Storage, Graph Storage, KV Storage), đáp ứng các yêu cầu khác nhau về quy mô và loại dữ liệu.

4.1.2. Quy trình Truy vấn và Sinh câu trả lời (Querying and Generation Pipeline)

- **Retrievers**: Kết hợp với VectorDB, truy xuất thông tin liên quan dựa trên câu hỏi. Chất lượng phụ thuộc vào mô hình nhúng và tham số như *k chunks*.
- **Retrievers trong LightRAG tận dụng cả tìm kiếm vector và truy vấn đồ thị tri thức thông qua các Query Modes đa dạng (Local, Global, Hybrid, Naive, Mix Mode)**, cho phép truy xuất thông tin một cách thông minh và có ngữ cảnh hơn.
- **ReRankers (Tùy chọn)**: Cải thiện chất lượng chunks bằng mô hình đánh giá phức tạp, giảm nhiễu, tăng độ chính xác cho LLM.
- **Prompt Builders / Formatters**: Tạo prompt hiệu quả thông qua *PromptTemplate*, đảm bảo LLM sử dụng ngữ cảnh đúng cách. Thiết kế prompt là yếu tố then chốt.
- Prompt Builders được hỗ trợ bởi các định nghĩa ENTITY_TYPES và các hàm context_support để tạo ra các prompt giàu ngữ cảnh, trong khi Generators (LLMs) được tích hợp các wrapper như *gemini_complete_vietnamese* để đảm bảo phản hồi chính xác và tự nhiên bằng tiếng Việt.

4.1.3. Evaluators (Thành phần Đánh giá)

- Đánh giá hiệu suất RAG cần thiết để đo lường chất lượng. LightRAG nên hỗ trợ các chỉ số như Context Precision/Recall, Faithfulness, Answer Relevance, tích hợp với công cụ như RAGAs.

LightRAG cung cấp kiến trúc module hóa, chuẩn hóa quy trình RAG, với mỗi thành phần đóng vai trò quan trọng, phối hợp nhịp nhàng để đạt hiệu quả cao.

4.2. Đánh giá chung về ưu điểm, nhược điểm của LightRAG

Ưu điểm:

- **Đơn giản hóa phát triển**: LightRAG trừu tượng hóa chi tiết kỹ thuật, cung cấp thành phần dựng sẵn, giảm thời gian và công sức xây dựng pipeline RAG.
- **Khả năng hiểu mối quan hệ phức tạp**: Nhờ kiến trúc đồ thị tri thức, LightRAG có thể truy vấn và tổng hợp thông tin dựa trên các mối liên hệ ngữ nghĩa sâu sắc giữa các thực thể, vượt xa khả năng tìm kiếm từ khóa hoặc vector đơn thuần.
- **Tối ưu hóa cho tiếng Việt**: Prompts và các loại thực thể được thiết kế riêng cho ngôn ngữ tiếng Việt đảm bảo chất lượng phản hồi cao và văn phong tự nhiên.
- **Các chế độ truy vấn đa dạng**: Hỗ trợ nhiều chiến lược truy vấn khác nhau (Local, Global, Hybrid, Naive, Mix Mode) giúp nhà phát triển linh hoạt lựa chọn phương pháp phù hợp nhất với từng loại câu hỏi.
- **Tính module hóa, linh hoạt**: Cho phép tùy chọn và thay thế thành phần (Embedder, VectorDB, LLM) phù hợp với bài toán.
- **Tập trung vào nghiệp vụ**: Giúp nhà phát triển chú trọng vào dữ liệu, prompt và trải nghiệm người dùng.
- **Tích hợp công nghệ tiên tiến**: Hỗ trợ các mô hình nhúng, LLM, VectorDB mới nhất.
- **Khả năng mở rộng**: Thiết kế tốt cho phép thêm module tùy chỉnh hoặc tích hợp công cụ khác.
- **Giảm "ảo giác" LLM**: Cung cấp ngữ cảnh cụ thể, tăng độ chính xác câu trả lời.

- **Quản lý tài liệu nâng cao**: Khả năng theo dõi trạng thái xử lý tài liệu (PENDING, PROCESSING, PROCESSED, FAILED) và các chức năng xóa theo ID tài liệu hoặc tên thực thể.

Nhược điểm:

- **Độ phức tạp tiềm ẩn**: Cấu hình tối ưu đòi hỏi kiến thức về RAG.
- **Độ phức tạp trong việc xây dựng và quản lý đồ thị tri thức**: Mặc dù LightRAG cung cấp công cụ, nhưng việc thiết kế schema KG, đảm bảo chất lượng trích xuất thực thể/mối quan hệ và tối ưu hóa truy vấn KG vẫn là một thách thức.
- **Phụ thuộc vào dữ liệu và prompt**: Chất lượng đầu ra phụ thuộc vào dữ liệu đầu vào và kỹ năng prompt engineering.
- **Thách thức đánh giá**: Đánh giá RAG phức tạp, cần công cụ mạnh hơn.
- **Chi phí vận hành**: API của mô hình nhúng và LLM (như Gemini).
- **Đường cong học tập**: Người mới cần thời gian làm quen với các khái niệm và thành phần.
- **Tùy biến sâu hạn chế**: Trừu tượng hóa có thể giới hạn can thiệp sâu cho yêu cầu đặc thù.

4.3. Thách thức khi làm việc với LightRAG và hệ thống RAG

- **Chuẩn bị dữ liệu**: Thu thập, làm sạch và cấu trúc hóa dữ liệu (đặc biệt từ PDF phức tạp) tốn thời gian. Chiến lược chunking cần tối ưu để giữ ngữ nghĩa.
- **Trích xuất Thực thể và Mối quan hệ**: Đảm bảo độ chính xác của quá trình Entity Identification và Relationship Mapping từ văn bản tiếng Việt là rất quan trọng.

- **Mô hình nhúng**: Chọn mô hình phù hợp với ngôn ngữ và lĩnh vực, hiệu suất thay đổi theo dữ liệu.
- **VectorDB**: Lựa chọn và cấu hình chỉ mục cân bằng tốc độ và độ chính xác.
- **Tối ưu hóa Truy vấn Đồ thị**: Lựa chọn Query Mode phù hợp và tối ưu hóa các thao tác trên đồ thị tri thức để đạt hiệu suất mong muốn.
- **Prompt Engineering**: Thiết kế prompt hiệu quả đòi hỏi thử nghiệm và kinh nghiệm, cần xử lý "ảo giác" LLM.
- **Đánh giá và gỡ lỗi**: Đánh giá toàn diện phức tạp, xác định lỗi (chunking, embedding, retrieval, prompting) cần công cụ hỗ trợ.
- **Cân bằng chính xác, tốc độ, chi phí**: Mô hình lớn tăng độ chính xác nhưng làm tăng độ trễ và chi phí.
- **Cập nhật kho tri thức**: Cần quy trình hiệu quả để re-index khi dữ liệu thay đổi.

4.4. Đề xuất hướng phát triển (đối với LightRAG framework)

Để LightRAG tiếp tục phát triển và trở thành một framework mạnh mẽ hơn nữa, các hướng phát triển sau đây được đề xuất:

- **Tiền xử lý dữ liệu thông minh**:
    - Tích hợp công cụ xử lý cấu trúc phức tạp (bảng, hình ảnh) trong tài liệu.
    - Phát triển chunking dựa trên ngữ nghĩa hoặc cấu trúc logic.
- **Phát triển và tối ưu hóa mô hình trích xuất KG**: Nghiên cứu các mô hình NER (Named Entity Recognition) và RE (Relation Extraction) chuyên biệt cho tiếng Việt và lĩnh vực lịch sử để cải thiện độ chính xác.
- **Tối ưu Retriever**:

    - Hỗ trợ Hybrid Search và ReRanking mạnh mẽ.
    - Áp dụng truy xuất đa bước hoặc dựa trên đồ thị tri thức.
- **Mở rộng và cải thiện Graph Storage backend**: Tích hợp thêm các giải pháp Graph Database mạnh mẽ hơn và tối ưu hóa hiệu suất truy vấn.
- **Cải thiện Prompt Engineering**:
    - Cung cấp giao diện thử nghiệm và quản lý prompt.
    - Tích hợp tối ưu hóa prompt tự động.
- **Tích hợp và tinh chỉnh LLM**:
    - Hỗ trợ LLM mới nhất.
    - Cung cấp tiện ích tinh chỉnh LLM nhỏ cho dữ liệu chuyên ngành.
- **Nghiên cứu các chiến lược Query Modes nâng cao**: Khám phá các phương pháp kết hợp truy vấn vector và truy vấn đồ thị một cách thông minh hơn.
- **Tối ưu hóa các wrapper LLM cho tiếng Việt**: Tiếp tục cải thiện các wrapper để đảm bảo ngữ điệu, văn phong và sự tinh tế trong câu trả lời tiếng Việt.
- **Hệ thống đánh giá**:
    - Tích hợp chỉ số RAG (RAGAs) và trực quan hóa kết quả.
    - Hỗ trợ thu thập phản hồi người dùng.
- **Trải nghiệm nhà phát triển**:
    - Cung cấp tài liệu chi tiết, ví dụ thực tế, công cụ gỡ lỗi.
- **Triển khai và quản lý**:
    - Hỗ trợ đóng gói và triển khai ứng dụng.

    - Quản lý phiên bản dữ liệu, chỉ mục, mô hình.

4.5. Hướng phát triển đề tài

Đề tài "Ứng dụng RAG kết hợp LLMs trong xây dựng chatbot Lịch sử" có thể phát triển theo các hướng dưới đây để hoàn thiện và nâng cao giá trị ứng dụng:

- **Mở rộng nguồn tri thức**:
    - Bao gồm sách giáo khoa lớp 11, 12 và tài liệu tham khảo uy tín.
    - Xử lý dữ liệu đa phương tiện (hình ảnh, video tư liệu).
- **Làm giàu đồ thị tri thức lịch sử Việt Nam**: Tự động hoặc bán tự động bổ sung thêm các thực thể và mối quan hệ từ các nguồn dữ liệu lịch sử phong phú hơn, bao gồm cả dữ liệu bán cấu trúc và phi cấu trúc.
- **Nâng cao tương tác**:
    - Quản lý hội thoại để ghi nhớ ngữ cảnh.
    - Cá nhân hóa câu trả lời theo trình độ học sinh.
    - Đặt câu hỏi ngược để khuyến khích tư duy.
- **Tận dụng tối đa các chế độ truy vấn của LightRAG**: Thử nghiệm các Query Modes khác nhau để xem chế độ nào mang lại kết quả tốt nhất cho các loại câu hỏi lịch sử đa dạng.
- **Phát triển khả năng suy luận trên đồ thị tri thức**: Cho phép chatbot trả lời các câu hỏi suy luận phức tạp hơn dựa trên mối quan hệ giữa các thực thể trong KG.
- **Đánh giá nâng cao**:
    - Xây dựng bộ dữ liệu ground truth cho Lịch sử Việt Nam.
    - Triển khai đánh giá tự động và thu thập phản hồi từ học sinh, giáo viên.

- **Thử nghiệm và so sánh**:
    - So sánh mô hình nhúng, VectorDB, và LLM khác nhau.
- **Giao diện người dùng**:
    - Xây dựng giao diện web/di động thân thiện.
    - Hiển thị nguồn thông tin để tăng độ tin cậy.
- **Kỹ thuật RAG tiên tiến**:
    - Áp dụng ReAct, Self-Correction hoặc RAG phức tạp hơn.

Việc phát triển theo các hướng này sẽ hoàn thiện chatbot Lịch sử, đóng góp vào ứng dụng AI trong giáo dục, mang lại công cụ học tập hiện đại và hữu ích.

TÀI LIỆU THAM KHẢO

1. Guo, Z., Xia, L., Yu, Y., Ao, T., & Huang, C. (2024). *LightRAG: Simple and Fast Retrieval-Augmented Generation*. arXiv preprint arXiv:2410.05779.

2. Lewis, P., Liu, Y., Priyなどで、Xu, J., & Fasoli, A. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. arXiv preprint arXiv:2005.11477.

3. Ehrlinger, L., & Wöß, W. (2016). *Towards a definition of knowledge graphs*. In Proceedings of the 2016 International Conference on Semantic Web and Web Services (SWWS) (pp. 9-18). The Steering Committee of The World Congress in Computer Science, Computer Engineering, and Applied Computing (WorldComp).

4. Jaykumaran. *LightRAG: Simple and Fast Alternative to GraphRAG*

5. HKUDS. *GitHub - HKUDS/LightRAG: "LightRAG: Simple and Fast Retrieval-Augmented Generation"*

6. Trung Đức. *GraphRAG - Một sự nâng cấp mới của RAG truyền thống chăng?*

7. Shrinivasan Sankar. *LightRAG - Is It a Simple and Efficient Rival to GraphRAG? | HackerNoon*